Universidad Rafael Landívar de Guatemala
Facultad de Ingeniería
Ingeniería de Software I
Ing. Jorge Escalante Castañeda
2026



AgroCloud

Manual Técnico

Alumnos:
Afre Reyes, Diego Alejandro Federico – 1594422
Ixquiac Sac, Luis David – 1521223
Ovalle Pedroza, Diego Alejandro – 1500324
Ramirez Alvarez, Javier Estuardo – 1647124

Quetzaltenango, Guatemala, 31 de octubre de 2026

CONTENIDO

1. INTRODUCCIÓN 7

2. PROPÓSITO 11

3. ALCANCE DEL SISTEMA 13

3.1. Componentes del sistema 13

3.1.1. Cliente web 13

3.1.2. Servidor de aplicaciones y API REST 13

3.1.3. Base de datos y persistencia 13

3.1.4. Infraestructura y servicios externos 14

3.2. Funcionalidades implementadas 14

3.2.1. Gestión de identidad, sesiones y seguridad criptográfica 14

3.2.2. Catálogo de planes y suscripciones agrícolas 14

3.2.3. Motor de aprovisionamiento dinámico de bases de datos 15

3.2.4. Telemetría y monitoreo de recursos en tiempo real 15

3.2.5. Sistema de mesa de ayuda e incidencias técnicas 16

3.2.6. Consola de administración global 16

4. ESTADO DEL SISTEMA 17

4.1. Capacidades operativas 17

4.2. Limitaciones conocidas 17

4.2.1. Simulación del proceso de pago 17

4.2.2. Telemetría de hardware del sistema host 17

4.2.3. Ausencia de interfaz OpenAPI/Swagger integrada 17

4.2.4. Alcance exclusivo web 18

5. ARQUITECTURA DEL SISTEMA 19

5.1. Modelo arquitectónico global 19

5.2. Flujo de una solicitud de extremo a extremo 20

5.3. Responsabilidad de cada capa 21

5.3.1. Capa de presentación (Frontend) 21

5.3.2. Capa de seguridad y filtros (Spring Security) *22*
5.3.3. Capa lógica de negocio y domino (Backend service) *22*
5.3.4. Capa de acceso a datos y persistencia (PostgreSQL) *23*
5.3.5. Capa de servicios externos de notificación *23*

6. TECNOLOGÍAS Y REQUISITOS **24**

6.1. Tecnologías y versiones reales del proyecto **24**

6.1.1. Ecosistema Backend *24*
6.1.2. Ecosistema Frontend *25*
6.1.3. Base de datos e infraestructura *26*

6.2. Requisitos del entorno de desarrollo y ejecución **26**
6.3. Requisitos de hardware y sistema operativo **27**

6.3.1. Sistemas operativos compatibles *27*
6.3.2. Especificaciones de hardware *27*

7. CONFIGURACIÓN Y EJECUCIÓN **28**

7.1. Requisitos comprobados en el proyecto **28**
7.2. Preparación de una copia de desarrollo **28**
7.3. Ejecución completa con Docker Compose **29**
7.4. Backend y Frontend en terminales separadas **29**
7.5. Variables de configuración **30**
7.6. Compilación y verificación del arranque **31**

8. BASE DE DATOS **32**

8.1. Modelo de persistencia y relaciones **32**
8.2. Migraciones y estado final del esquema **33**
8.3. Diccionario de datos de la base de control **34**

8.3.1. Users *34*
8.3.2. Roles *35*
8.3.3. Service_plans *35*
8.3.4. Subscriptions *35*
8.3.5. Instances *36*

8.3.6. Agrocloud_plantillas *37*
8.3.7. Incidents *37*
8.3.8. Notifications *38*
8.3.9. Indicent_comments *39*
8.3.10. Payments *39*
8.3.11. Invoices *40*
8.3.12. Instance_metrics *40*
8.4. Integridad referencial e índices **41**
8.5. Esquemas agrícolas y tablas personalizadas **41**
9. API REST **43**
9.1. Convenciones y autenticación **43**
9.2. Catálogo de operaciones **43**
9.2.1. AdminUserController *43*
9.2.2. AuthController *44*
9.2.3. IncidentController *44*
9.2.4. InstanceController *46*
9.2.5. NotificationController *47*
9.2.6. InvoiceController *47*
9.2.7. PaymentController *48*
9.2.8. PlanController *48*
9.2.9. SubscriptionController *49*
9.2.10. TemplateController *50*
9.3. Contratos de entrada y salida **50**
9.4. Respuestas de error **54**
10. MÓDULOS Y REGLAS DE NEGOCIO **56**

10.1. Usuarios y cuentas 56
10.2. Planes y complementos 56
10.3. Suscripciones y contratación 57
10.4. Instancias y aprovisionamiento de esquemas 58
10.5. Explorador de tablas y datos 59
10.6. Dependencias con incidentes y facturación 59
10.7. Evidencias de implementación y límites de la versión 60
11. FACTURACIÓN, PAGOS Y MÉTRICAS DE RECURSOS 61
11.1. Monitoreo de recursos del sistema 61
11.1.1. Consulta de métricas de recursos 61
11.2. Historial de pagos 62
11.2.1. Consulta de transacciones 62
11.3. Consulta de facturas 62
11.3.1. Visualización y descarga de facturas 63
11.4. Auditoría de transacciones financieras 63
11.4.1. Supervisión de pagos 63
12. ERRORES Y PREGUNTAS FRECUENTES 64
12.1. Diagnóstico y resolución de errores 64
12.1.1. Consulta de métricas de recursos 64
12.1.2. Errores de validación de contraseñas 65
12.1.3. Errores de permisos y acceso 65
12.1.4. Límite de instancias alcanzado 65
12.1.5. Errores de conectividad externa 66
12.1.6. Expiración del enlace de verificación 66
12.1.7. Indisponibilidad temporal de instancias 66
12.2. Preguntas frecuentes sobre planes y almacenamiento 67
12.2.1. Planes disponibles 67
12.2.2. Límite de almacenamiento 67

12.2.3. Consulta de métricas de recursos *68*

12.3. Preguntas frecuentes sobre conectividad **68**

12.3.1. Conexión de aplicaciones externas *68*

12.3.2. Consulta de credenciales de conexión *69*

13. SEGURIDAD PARA USUARIOS Y MESA DE SOPORTE **70**

13.1. Recomendaciones de seguridad **70**

13.1.1. Protección de credenciales de base de datos *70*

13.1.2. Seguridad de contraseñas de usuario *71*

13.1.3. Cierre de sesión en equipos compartidos *71*

13.2. Atención y resolución de incidencias **71**

13.2.1. Protección de credenciales de base de datos *72*

13.2.2. Resolución y cierre de incidencias *72*

CONTENIDO DE FIGURAS

Figura 1. Modelo del dominio completo 73
Figura 2. Clases de servicios y colaboradores 74
Figura 3. AgroCloud - Acceso de cliente 75
Figura 4. AgroCloud - Acceso de clientes a servicios 76
Figura 5. AgroCloud - Acceso de soporte técnico a modales 77
Figura 6. AgroCloud - Funciones de administrador 77
Figura 7. Registro y contratación de servicios 78
Figura 8. Creación de una instancia 79
Figura 9. Atención de una incidencia por parte de soporte técnico 80
Figura 10. Inicio de sesión 81
Figura 11. Aprovisionamiento de instancia 81
Figura 12. Soporte y notificaciones 82
Figura 13. Transacciones principales de una instancia 82
Figura 14. Estados de una incidencia 82
Figura 15. Componentes principales del sistema 83
Figura 16. Despliegue en Docker 83
Figura 17. Directorio del backend de AgroCloud 84
Figura 18. Modelo Entidad-Relación 85

1. INTRODUCCIÓN

AgroCloud es una plataforma web orientada a la administración de bases de datos para el sector agrícola. Su desarrollo responde a la necesidad de disponer de herramientas que permitan organizar información, administrar recursos tecnológicos y facilitar el acceso a servicios de almacenamiento sin que cada usuario tenga que configurar por su cuenta toda la infraestructura. La plataforma reúne funciones de registro, contratación de planes, creación de instancias, consulta de métricas y atención de incidencias dentro de un mismo entorno.

En las actividades agrícolas se generan datos relacionados con cultivos, parcelas, producción, ganadería y otras operaciones que requieren un almacenamiento ordenado. Cuando esa información se maneja en archivos dispersos o sistemas aislados, su consulta y administración pueden complicarse. AgroCloud propone una alternativa basada en el modelo de base de datos como servicio (DBaaS), mediante el cual los usuarios pueden solicitar y administrar recursos de base de datos desde una aplicación web. Su enfoque no consiste en sustituir las herramientas agrícolas especializadas, sino en proporcionar una base tecnológica para almacenar y gestionar información de distintos procesos productivos.

El funcionamiento de AgroCloud se apoya en una arquitectura que separa la interfaz de usuario, la lógica de negocio y la persistencia de datos. La interfaz web permite interactuar con las funciones disponibles según el perfil de acceso. El servidor procesa las solicitudes, aplica las reglas de negocio y controla las operaciones autorizadas. PostgreSQL almacena tanto la información general de la plataforma como los esquemas creados para las instancias de los clientes. Esta separación facilita comprender las responsabilidades de cada componente y localizar los puntos donde se realizan las validaciones y operaciones principales.

La aplicación contempla distintos perfiles de usuario. El cliente agrícola dispone de opciones para consultar su plan, administrar instancias y solicitar asistencia. El administrador supervisa usuarios, servicios y recursos generales, mientras que el personal de soporte atiende incidencias y revisa información técnica asociada con las instancias. Estas funciones se complementan con mecanismos de autenticación y autorización que restringen el acceso según los

permisos asignados. De esa manera, las operaciones sensibles no dependen únicamente de los controles visibles en la interfaz, sino también de las verificaciones realizadas por el servidor.

Uno de los elementos centrales de AgroCloud es el aprovisionamiento de bases de datos mediante esquemas de PostgreSQL. Al solicitar una instancia, el sistema verifica las condiciones de la suscripción y ejecuta las operaciones necesarias para crear la estructura correspondiente. Las plantillas agronómicas permiten preparar tablas relacionadas con distintos tipos de actividad agrícola. Además, el sistema incorpora mecanismos para gestionar credenciales de conexión, consultar el estado de las instancias y mostrar información sobre el consumo de recursos. Estos procesos requieren una coordinación clara entre los servicios del backend, los repositorios y el motor de base de datos.

La seguridad forma parte del diseño técnico de la plataforma. El acceso a las operaciones de la API se controla mediante autenticación con tokens JWT y permisos asociados a los roles definidos. Las credenciales de las bases de datos reciben un tratamiento específico mediante cifrado, y los flujos de verificación de correo y recuperación de contraseñas contribuyen a la administración de cuentas. También se contemplan servicios de notificación y comunicación con proveedores externos, cuya configuración depende de variables de entorno y de las condiciones de ejecución de cada instalación.

Para facilitar el desarrollo y la operación, AgroCloud utiliza tecnologías diferenciadas para cada responsabilidad: React y TypeScript en el cliente web; Java y Spring Boot en el servidor; PostgreSQL para la persistencia; y Docker para la ejecución de servicios en contenedores. La organización del repositorio distingue los proyectos de frontend y backend, junto con los archivos de configuración, los recursos de despliegue y los materiales de apoyo para pruebas. Conocer esta distribución es importante para instalar el sistema, identificar dependencias, revisar cambios y realizar tareas de mantenimiento sin afectar componentes ajenos.

El manual técnico reúne la descripción de la arquitectura, las tecnologías, los requisitos del entorno y la estructura del código fuente. También delimita las funciones implementadas y las restricciones identificadas, como la simulación del procesamiento de pagos y el alcance de las

métricas de recursos. Esta información sirve como referencia para quienes desarrollan, despliegan, evalúan o mantienen AgroCloud, y permite relacionar las decisiones de diseño con los componentes que intervienen en su funcionamiento. La descripción técnica se centra en las capacidades documentadas del proyecto y distingue las funciones operativas de aquellas que requieren integraciones adicionales.

2. PROPÓSITO

Proporcionar información técnica detallada sobre la estructura, funcionamiento y configuración de AgroCloud, facilitando la comprensión de los componentes que conforman la plataforma y los procesos que intervienen en su operación. Se busca establecer una referencia clara sobre la arquitectura del sistema, las tecnologías utilizadas, la organización del código fuente y los mecanismos de comunicación entre el frontend, backend y la base de datos.

Facilitar las actividades de instalación, configuración, mantenimiento y actualización del sistema mediante la descripción de los requisitos de software y hardware, las dependencias necesarias para su ejecución y la distribución de los archivos del proyecto. Esta información permite identificar los recursos necesarios para preparar un entorno de desarrollo y comprender las responsabilidades de cada componente dentro de la plataforma.

Describir los procesos internos relacionados con la administración de usuarios, autenticación, control de acceso, gestión de suscripciones y aprovisionamiento de bases de datos. También se busca explicar los mecanismos de seguridad, almacenamiento de información y comunicación con servicios externos, permitiendo comprender cómo se ejecutan las principales operaciones y cómo interactúan los diferentes módulos para garantizar el funcionamiento del sistema.

Establecer una base de consulta para desarrolladores, administradores de sistemas y personal de soporte técnico que participen en la administración o mantenimiento de AgroCloud. La información técnica permite identificar los módulos responsables de cada funcionalidad, comprender sus dependencias y facilitar la detección de errores, la implementación de mejoras y la incorporación de nuevas características.

Mantener una referencia organizada de las decisiones técnicas y los criterios de desarrollo utilizados en la construcción de la plataforma, favoreciendo la continuidad del proyecto y el trabajo colaborativo. La documentación de los componentes, sus responsabilidades y sus relaciones

contribuye a reducir la dependencia del conocimiento individual de los desarrolladores y facilita futuras modificaciones, procurando conservar la estabilidad, seguridad y organización del sistema.

3. ALCANCE DEL SISTEMA

AgroCloud incluye los componentes y las funciones que se describen a continuación. Su alcance comprende la administración web de usuarios, suscripciones e instancias de bases de datos, así como las operaciones de soporte y supervisión asociadas.

3.1. Componentes del sistema

La plataforma AgroCloud se compone de cuatro subsistemas principales desacoplados que interactúan a través de protocolos web estándar:

3.1.1. Cliente web

Desarrollado en React 19 con TypeScript, empaquetado y servido mediante Vite, y estilizado con Tailwind CSS v4. Provee una interfaz responsiva y reactiva adaptada a cuatro experiencias de usuario: Visitante (Landing/Registro), Cliente Agrícola (ClientPanel), Administrador de Infraestructura (AdminPanel) y Agente de Soporte Técnico (SupportPanel).

3.1.2. Servidor de aplicaciones y API REST

Desarrollado sobre el ecosistema Java 21 / Spring Boot (con spring-boot-starter-parent 4.1.1/3.x). Implementa la lógica de negocio orientada a dominios, control de acceso stateless mediante tokens JWT, cifrado criptográfico simétrico, gestión reactiva de eventos y aprovisionamiento DDL/DML automatizado.

3.1.3. Base de datos y persistencia

Basado en PostgreSQL 16. Opera bajo un enfoque híbrido:

- Esquema Core (public): Gestiona las entidades maestras de la plataforma (usuarios, roles, planes de suscripción, transacciones, tickets de soporte y métricas).

- Esquemas Multi-Tenant Aislados (inst_<nombre>_<hash>): Esquemas dedicados aprovisionados en caliente para cada cliente, conteniendo las tablas agronómicas de la plantilla seleccionada.

3.1.4. Infraestructura y servicios externos

- Orquestación de contenedores mediante Docker y Docker Compose.
- Integración de pasarela transaccional de correo para verificación de cuentas y restablecimiento de claves mediante la API HTTPS de Brevo y soporte alternativo vía SMTP.

3.2. Funcionalidades implementadas

El sistema cuenta con las siguientes capacidades operativas verificadas en el código fuente:

3.2.1. Gestión de identidad, sesiones y seguridad criptográfica

Registro público de cuentas con verificación obligatoria mediante enlace de correo electrónico de un solo uso (token criptográfico SHA-256 de 256 bits, vigencia de 24 horas).

Flujo seguro de recuperación de contraseña con invalidación retroactiva de tokens JWT previos mediante versionado de sesión (auth_version).

Autenticación sin estado basada en JWT (JSON Web Tokens) con firma criptográfica HMAC y control de acceso basado en roles (ADMINISTRADOR, CLIENTE, SOPORTE).

3.2.2. Catálogo de planes y suscripciones agrícolas

Contratación y administración de cuatro niveles de servicio: Finca (10 GB), Productor (50 GB), Agro Pro (100 GB) y Agro Enterprise (250 GB).

Control de cuotas de recursos: validación de límite de instancias concurrentes (maxInstances) y almacenamiento asignado antes de autorizar aprovisionamientos.

Ciclo de vida completo de suscripciones: estados pending_payment, active, suspended y cancelled.

3.2.3. Motor de aprovisionamiento dinámico de bases de datos

Aprovisionamiento en caliente mediante InstanceSchemaProvisioningService: ejecución real de CREATE SCHEMA y despliegue del DDL/DML correspondiente a plantillas agronómicas (Cultivos y parcelas, Ganadería y pastos, Hortalizas, Granos básicos).

Generación aleatoria y cifrado simétrico robusto (AES-256 / GCM / NoPadding) de credenciales maestras de base de datos (db_password_encrypted), asegurando que las contraseñas nunca se almacenen ni viajen en texto plano.

Exposición de credenciales bajo demanda mediante endpoint protegido y aislado exclusivamente al cliente propietario (GET /api/v1/instancias/{id}/credenciales).

Conectividad externa directa a las instancias a través del puerto expuesto del motor (15432 en host local).

3.2.4. Telemetría y monitoreo de recursos en tiempo real

Registro y consulta histórica de métricas en la tabla instance_metrics: consumo de CPU (%), utilización de memoria RAM (%) y medición real del almacenamiento ocupado por el esquema PostgreSQL (calculateSchemaStorageGb).

Sondeo periódico y visualización reactiva en dashboards mediante indicadores de salud y estado operativo (active, revision, suspended, terminated).

3.2.5. Sistema de mesa de ayuda e incidencias técnicas

Flujo completo de tickets de soporte asociados a instancias específicas, asignación de números correlativos públicos (ticketNumber) y priorización.

Hilo de comentarios bidireccionales con visibilidad segmentada por rol (targetRole) entre el agricultor y los ingenieros de soporte.

Notificaciones reactivas en tiempo real mediante Server-Sent Events (SSE) vía NotificationStream, alertando inmediatamente sobre cambios de estado o nuevos comentarios.

3.2.6. Consola de administración global

Monitoreo consolidado del estado de salud del clúster de bases de datos.

Mantenimiento de cuentas de usuarios: activación, suspensión preventiva y alta administrativa de personal técnico.

4. ESTADO DEL SISTEMA

4.1. Capacidades operativas

Las funciones implementadas incluyen el registro y autenticación de usuarios, la gestión de planes, el aprovisionamiento de esquemas PostgreSQL, la consulta de métricas, el seguimiento de incidencias y las opciones administrativas. Su disponibilidad depende de la configuración correcta de los servicios y del entorno de ejecución.

4.2. Limitaciones conocidas

Las siguientes restricciones delimitan las funciones disponibles y las condiciones de ejecución de AgroCloud:

4.2.1. Simulación del proceso de pago

La plataforma implementa validación sintáctica de tarjetas y generación formal de comprobantes contables (payments e invoices), pero no se conecta a pasarelas bancarias reales (Stripe, PayPal, Visanet) para procesar cobros monetarios reales, manteniéndose dentro del alcance académico convenido.

4.2.2. Telemetría de hardware del sistema host

Los valores de CPU y RAM de las instancias reflejan la carga transaccional simulada y el estado snapshot de la base de datos; no se emplean agentes SNMP de bajo nivel ni daemons de Linux (como Prometheus node_exporter) instalados directamente en el kernel del host físico.

4.2.3. Ausencia de interfaz OpenAPI/Swagger integrada

El backend no incluye dependencias activas de springdoc-openapi ni interfaces gráficas tipo Swagger UI; la especificación formal de la API se gestiona mediante la colección oficial de Postman (AgroCloud_Postman_Collection.json) y contratos DTO.

4.2.4. Alcance exclusivo web

No existe soporte ni aplicaciones nativas compiladas para iOS o Android; el acceso se encuentra optimizado para clientes web de escritorio y móviles mediante navegadores modernos compatibles con HTML5 y ES2022.

5. ARQUITECTURA DEL SISTEMA

5.1. Modelo arquitectónico global

AgroCloud utiliza una arquitectura de software basada en tres capas principales: presentación, lógica de negocio y persistencia. Esta organización permite distribuir las responsabilidades del sistema, facilitando su desarrollo, mantenimiento y escalabilidad. Cada capa cumple una función específica y se comunica con las demás mediante mecanismos definidos, evitando que la interfaz de usuario dependa directamente de los procesos internos o de la estructura de la base de datos.

La capa de presentación corresponde a la interfaz web desarrollada con React y TypeScript, utilizando Vite como herramienta de desarrollo y compilación. Su función es permitir que los usuarios interactúen con las distintas funcionalidades de AgroCloud mediante los paneles de cliente, administrador y soporte técnico. Desde esta capa se realizan solicitudes HTTP a la API REST del backend, utilizando Axios para gestionar la comunicación y enviar los tokens de autenticación cuando se requiere acceder a recursos protegidos.

La capa de lógica de negocio está desarrollada con Java y Spring Boot. Se encarga de procesar las solicitudes recibidas desde el frontend, aplicar las reglas de funcionamiento y coordinar las operaciones de los diferentes módulos. Entre sus responsabilidades se encuentran la autenticación de usuarios, administración de suscripciones, creación de instancias de bases de datos, monitoreo de recursos y gestión de incidencias. También incorpora Spring Security para validar los tokens JWT y controlar el acceso a las funcionalidades según el rol asignado a cada usuario.

La capa de persistencia utiliza PostgreSQL como sistema gestor de bases de datos. Su función es almacenar y administrar la información necesaria para el funcionamiento de AgroCloud, incluyendo usuarios, roles, planes, suscripciones, instancias y registros de soporte. La plataforma implementa un modelo multi-tenant basado en esquemas independientes, donde el esquema principal public almacena la información administrativa y los esquemas adicionales contienen las

estructuras de datos correspondientes a las instancias creadas por los clientes. Para gestionar el acceso a los datos se utilizan Spring Data JPA, Hibernate y JdbcTemplate.

Además de estas tres capas, AgroCloud incorpora componentes de seguridad y servicios externos que complementan su funcionamiento. Los filtros de Spring Security verifican las solicitudes antes de permitir su procesamiento, mientras que la integración con Brevo y SMTP permite enviar correos relacionados con la verificación de cuentas y recuperación de contraseñas. Asimismo, Docker y Docker Compose facilitan la configuración y ejecución de los servicios que forman parte de la plataforma.

La comunicación entre los componentes sigue un flujo definido: el usuario realiza una acción desde la interfaz web, el backend recibe y valida la solicitud, ejecuta las operaciones correspondientes y consulta o modifica la información almacenada en PostgreSQL. Finalmente, los resultados regresan al frontend para su visualización. Esta separación permite mantener una estructura organizada, mejorar el control de acceso y facilitar futuras modificaciones sin afectar directamente a todos los componentes del sistema.

5.2. Flujo de una solicitud de extremo a extremo

Para comprender la dinámica de interacción entre capas, se describe el ciclo de vida completo de una solicitud transaccional típica (por ejemplo, la creación y aprovisionamiento de una nueva base de datos):

[Navegador / React SPA]

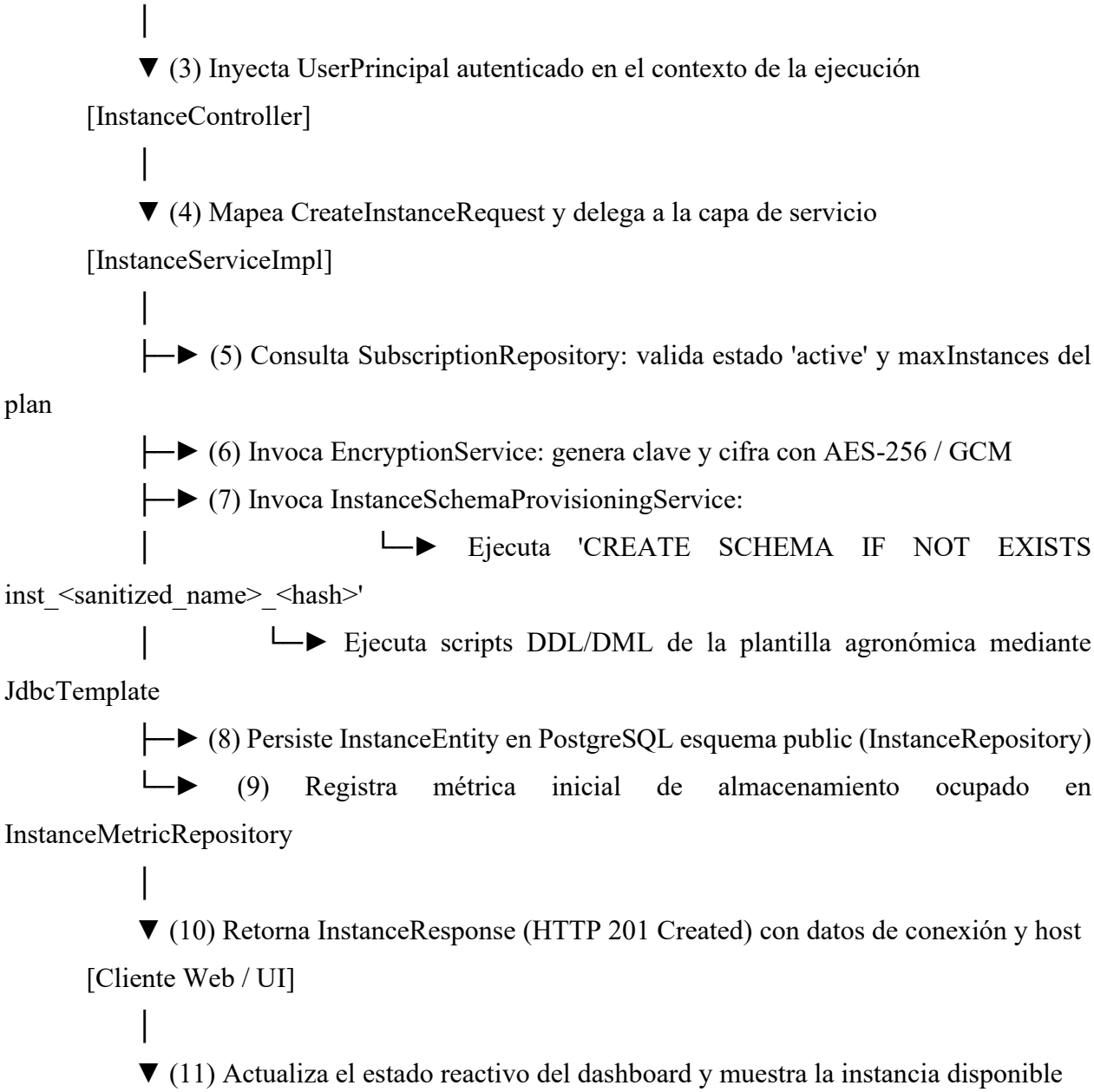
▼ (1) Petición HTTP POST /api/v1/instancias con Header 'Authorization: Bearer <JWT>'

[CORS & JwtAuthenticationFilter]



▼ (2) Valida firma del token, expiración y vigencia contra User.auth_version

[SecurityContext]



5.3. Responsabilidad de cada capa

5.3.1. Capa de presentación (Frontend)

Responsabilidad: Gestionar la interacción visual, renderizado condicional de rutas protegidas según el rol del usuario, validación de formularios en cliente y manejo centralizado del estado de sesión (AuthContext).

Manejo de Comunicaciones: Utiliza Axios configurado con interceptores de solicitud que adhieren automáticamente el encabezado Authorization: Bearer <token> y capturan respuestas 401 Unauthorized para forzar el cierre de sesión y redirección a la pantalla de acceso.

5.3.2. Capa de seguridad y filtros (Spring Security)

Responsabilidad: Interceptar cada solicitud HTTP entrante antes de alcanzar los controladores.

Mecanismos:

CorsFilter: Controla orígenes permitidos definidos por variable de entorno (CORS_ALLOWED_ORIGINS).

JwtAuthenticationFilter: Extrae el token Bearer, verifica su integridad criptográfica con la clave simétrica JWT_SECRET, valida los claims (sub, roles) y comprueba que el authVersion del token coincida con el registro del usuario en la base de datos (permitiendo invalidar sesiones de inmediato ante cambios de contraseña).

5.3.3. Capa lógica de negocio y domino (Backend service)

Responsabilidad: Implementar las reglas del negocio SaaS, transaccionalidad (@Transactional), control de cuotas de recursos y orquestación de servicios especializados.

Componentes Destacados:

InstanceServiceImpl: Coordina el ciclo de vida de las bases de datos.

InstanceSchemaProvisioningService: Traduce nombres de esquemas a identificadores SQL seguros y aprovisiona esquemas relacionales multi-tenant.

EncryptionService: Aplica cifrado y descifrado simétrico estándar militar mediante clave secreta inyectada.

NotificationStream: Administra un catálogo concurrente de SseEmitter mapeados por UUID de usuario para entrega de notificaciones en tiempo real sin polling.

5.3.4. Capa de acceso a datos y persistencia (PostgreSQL)

Responsabilidad: Almacenamiento durable, coherente y relacional bajo cumplimiento estricto de propiedades ACID.

Mecanismos:

Spring Data JPA / Hibernate: Abstracción ORM para operaciones sobre entidades del dominio principal (User, RoleEntity, InstanceEntity, SubscriptionEntity, etc.).

JdbcTemplate: Ejecución de consultas directas de bajo nivel para sentencias DDL estructuradas (CREATE SCHEMA, CREATE TABLE, COMMENT ON).

HikariCP: Administrador de pool de conexiones optimizado de alto rendimiento.

5.3.5. Capa de servicios externos de notificación

Responsabilidad: Enviar notificaciones transaccionales fuera de banda (enlaces de activación y tokens de restablecimiento).

Mecanismos: Módulo AccountMailService con estrategia dual: conexión HTTPS directa a la API v3 de Brevo (https://api.brevo.com/v3/smtp/email) cuando el proveedor configurado es brevo, o despacho sobre servidor SMTP tradicional.

6. TECNOLOGÍAS Y REQUISITOS

6.1. Tecnologías y versiones reales del proyecto

6.1.1. Ecosistema Backend

Tecnología / Biblioteca	Versión Implementada	Finalidad
Java Development Kit	OpenJDK 21 (LTS)	Entorno de ejecución y compilación de la lógica del servidor
Spring Boot Framework	4.1.1 / 3.4x	Framework base para el desarrollo de la API REST empresarial
Spring Security	Integrado en Spring Boot	Control de acceso, políticas CORS, CSRF disabled y seguridad stateless
JJWT (Java JWT)	0.12.6	Generación, firmado digital HMAC-SHA y parseo de tokens JWT
Spring Data JPA / Hibernate	Integrado en Spring Boot	Mapeo objeto-relacional y consultas de persistencia
Flyway Database Migrations	10.x / Flyway-Core	Control de versiones y ejecución secuencial de scripts DDL
PostgreSQL JDBC Driver	42.7.x	Conector de base de datos oficial para Java
Project Lombok	1.18.38	Reducción de código boilerplate
Apache Maven	3.9+ / Maven Wrapper	Gestor de dependencias y empaquetador del artefacto ejecutable

6.1.2. Ecosistema Frontend

Tecnología / Biblioteca	Versión Implementada	Finalidad
React	19.2.8	Biblioteca declarativa para construcción de componentes de interfaz
React DOM	19.2.8	Renderizador de React sobre el DOM web
TypeScript	6.0.2 / 5.x	Tipado estricto estático para modelos de datos, clientes API y vistas
Vite	8.3.0	Entorno de desarrollo rápido y bundler de producción ultraligero
Tailwind CSS	4.3.3	Framework de diesño utilitario para estilos CSS modernos y responsivos
Axios	1.20.0	Cliente HTTP para consumo de endpoints REST con interceptores de tokens
Lucide React	1.47.0	Paquete de iconografía vectorial técnica para dashboards y menús
Oxlint	1.81.0	Linter de alto rendimiento para validación estática de código TypeScript

6.1.3. Base de datos e infraestructura

Tecnología / Biblioteca	Versión Implementada	Finalidad
PostgreSQL Engine	16-alpine	Motor gestor relacional desplegado en contenedor oficial Docker
Docker Engine	24.0+	Plataforma de virtualización a nivel de sistema operativo
Docker Compose	v2.20+	Orquestador multi-contenedor para backend, frontend y PostgreSQL
Brevo Mail API	REST API v3	Pasarela transaccional en la nube para despacho seguro de correos

6.2. Requisitos del entorno de desarrollo y ejecución

Para compilar, ejecutar y mantener el sistema en una estación de trabajo técnica, se requiere contar con el siguiente software instalado y configurado en el PATH del sistema:

Java Development Kit: JDK 21 (o JDK 17 como versión mínima compatible). Comprobable mediante java -version.

Node.js & Gestor de Paquetes: Node.js versión 20.x LTS o superior (node -v) junto con npm versión 10.x+ (npm -v).

Motor Docker & Docker Compose: Docker Desktop (Windows/macOS) o Docker Engine + Compose Plugin (Linux) en estado activo.

Control de Versiones: Git 2.40+ configurado globalmente (git --version).

Entornos de Desarrollo Integrados (IDE) Recomendados:

Backend: IntelliJ IDEA Ultimate/Community o Eclipse IDE con plugins de Spring Tools.

Frontend: Visual Studio Code con extensiones oficiales de ESLint, Tailwind CSS IntelliSense y TypeScript.

6.3. Requisitos de hardware y sistema operativo

6.3.1. Sistemas operativos compatibles

Windows 10/11 (64-bit con soporte WSL2 activado), Ubuntu Server / Desktop 22.04 LTS o superior, macOS Monterey 12.0+ (procesadores Intel o Apple Silicon M-Series).

6.3.2. Especificaciones de hardware

Recurso de Hardware	Requisito Mínimo	Requisito Recomendado
Procesador (CPU)	Arquitectura x86_64	4 núcleos físicos o virtuales a 2.5 GHz o superior
Memoria RAM	8 GB de memoria total	16 GB de memoria RAM
Almacenamiento Libre	10 GB en disco SSD	25 GB en disco SSD NVMe
Conexión de Red	Acceso a internet	Conexión de banda ancha estable

7. CONFIGURACIÓN Y EJECUCIÓN

7.1. Requisitos comprobados en el proyecto

Componente	Requisito y fuente
Backend	Spring Boot 4.1.1 y nivel Java 21 en backend/pom.xml. El Dockerfile compila y ejecuta con imágenes Temurin 24.
Frontend	React 19.2.8, TypeScript 6.0.2 y Vite 8.3.0 declarados. El lockfile de Vite requiere Node ^20.19.0 o >=22.12.0; se recomienda Node 22.12 o superior.
Datos	PostgreSQL 16 en Docker Compose; Flyway administra las migraciones y Hibernate valida el esquema.
Herramientas	Git, Maven instalado y npm. Esta revisión no incluye mvnw ni mvnw.cmd; no se debe asumir un wrapper disponible.
Contenedores	Docker con Compose para la alternativa de ejecución completa. frontend/Dockerfile usa Node 22 y Nginx.

7.2. Preparación de una copia de desarrollo

Para reproducir la versión documentada, clonar el repositorio y seleccionar el commit de referencia. En una copia existente con cambios locales debe conservarse el trabajo antes de cambiar de revisión.

```
git clone https://github.com/Di3g0-01/Agrocloud.git
cd Agrocloud
git checkout 3f9ac6309e7fe3fd0fe85452966c12460c8155c2
Copy-Item .env.example .env
```

Editar .env con valores exclusivos del entorno local. Los marcadores de los ejemplos deben reemplazarse antes de iniciar. No copiar valores sensibles de application.yml ni de un despliegue compartido.

7.3. Ejecución completa con Docker Compose

docker-compose.e04.yml construye el backend y el frontend a partir del código. Usa una red y un volumen externos. Los archivos Compose principales comparten nombres de contenedores y puertos; elegir un solo conjunto por entorno.

```
docker network create agrocloud-net
docker volume create agrocloud-postgres-data
docker compose -f docker-compose.e04.yml up -d --build
docker compose -f docker-compose.e04.yml ps
```

Si la red o el volumen ya existen, reutilizarlos. El frontend queda en http://localhost:5173, la API en http://localhost:8080 y PostgreSQL en 127.0.0.1:15432. En la red interna la API usa db:5432. docker-compose.yml depende de una imagen local y de build-artifacts/app.jar; no constituye una compilación limpia desde el código actual.

7.4. Backend y Frontend en terminales separadas

La base puede iniciarse con el servicio db del Compose. Maven no importa automáticamente el archivo .env de la raíz: establecer las variables de la aplicación en la misma terminal que ejecutará el backend.

```
docker compose -f docker-compose.e04.yml up -d db
$env:DB_URL="jdbc:postgresql://localhost:15432/agrocloud_db"
$env:DB_USERNAME="<usuario_local>"
$env:DB_PASSWORD="<secreto_local>"
$env:JWT_SECRET="<clave_base64_de_al_menos_32_bytes>"
$env:CORS_ALLOWED_ORIGINS="http://localhost:5173"
$env:APP_FRONTEND_URL="http://localhost:5173"
cd backend
mvn spring-boot:run
```

En una segunda terminal, desde la raíz, preparar el frontend. VITE_API_URL incluye /api/v1 y se incorpora al compilado; si cambia, volver a compilar.

```
cd frontend
Set-Content .env.local "VITE_API_URL=http://localhost:8080/api/v1"
npm ci
npm run dev
```

7.5. Variables de configuración

Variable	Uso
POSTGRES_DB / POSTGRES_USER / POSTGRES_PASSWORD	Inicializan el contenedor PostgreSQL; deben coincidir con la conexión de la API.
DB_URL / DB_USERNAME / DB_PASSWORD	Conexión JDBC del backend. Fuera de Docker usar puerto 15432; dentro usar db:5432.
SERVER_PORT	Puerto HTTP, por defecto 8080.
JWT_SECRET / JWT_EXPIRATION	Firma JWT y duración, por defecto PT8H. Usar una clave propia.
CORS_ALLOWED_ORIGINS	Orígenes autorizados para las peticiones del navegador.
APP_FRONTEND_URL	Destino de enlaces de verificación y recuperación de cuenta.
MAIL_PROVIDER / MAIL_FROM	Seleccionan smtp o brevo y el remitente.
MAIL_HOST / MAIL_PORT	Servidor SMTP; localhost:1025 en ejecución local. En contenedor, localhost designa el propio contenedor.
MAIL_USERNAME / MAIL_PASSWORD / MAIL_SMTP_AUTH	Autenticación SMTP cuando el servidor la requiera.
MAIL_STARTTLS / MAIL_SSL	Opciones de cifrado de la conexión SMTP.
BREVO_API_KEY	Clave del proveedor HTTP, solo en backend cuando MAIL_PROVIDER=brevo.

HIKARI_MAX_POOL_SIZE / HIKARI_MIN_IDLE	Tamaño máximo y mínimo del pool JDBC; valores iniciales 5 y 1.
PUBLIC_DB_HOST / PUBLIC_DB_PORT / PUBLIC_DB_NAME	Destino alternativo que se devuelve para conexiones externas de instancias. No sustituye la conexión JDBC utilizada para crear los esquemas.
PUBLIC_DB_USER / PUBLIC_DB_PASSWORD	Credenciales alternativas de conexión pública. Configurarlas explícitamente y revisar sus privilegios.
VITE_API_URL	URL base del cliente Axios; por defecto http://localhost:8080/api/v1.

7.6. Compilación y verificación del arranque

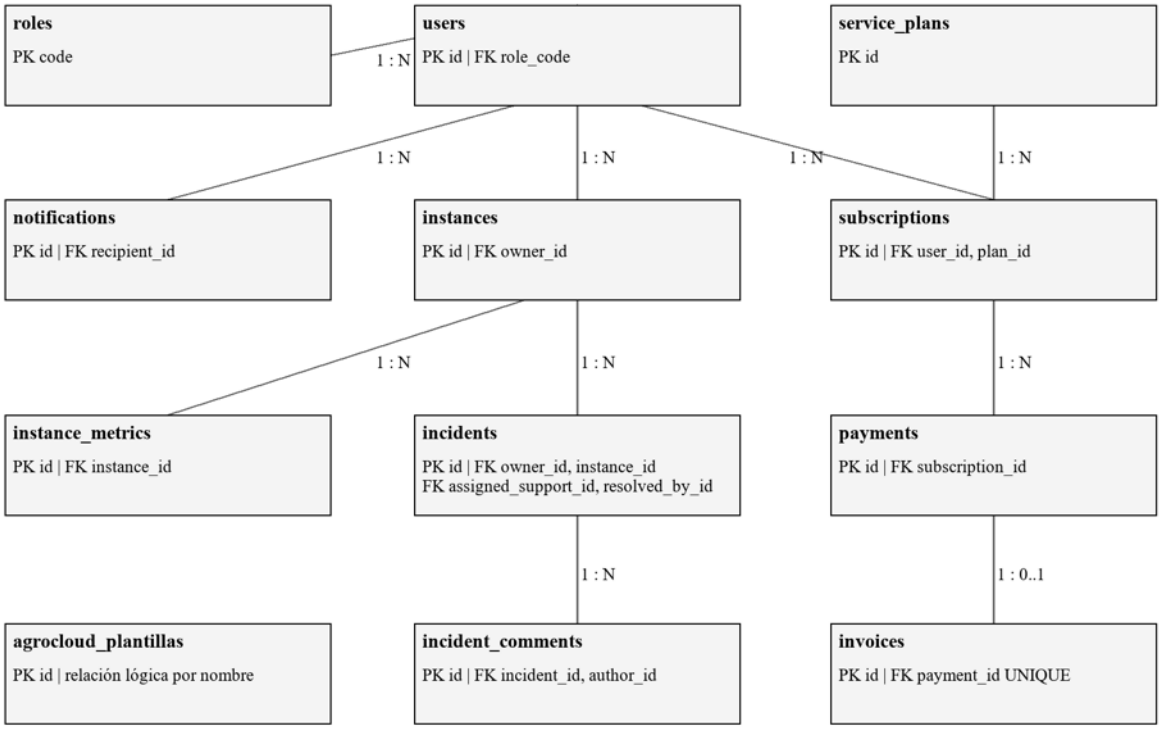
```
cd backend
mvn clean package -DskipTests
# En otra terminal desde la raíz:
cd frontend
npm run build
npm run preview
# Comprobar la API:
Invoke-RestMethod http://localhost:8080/actuator/health
```

El empaquetado del backend genera un JAR en target y la compilación del frontend genera dist. -DskipTests no ejecuta las pruebas. Un arranque correcto requiere que Flyway complete las migraciones y que Hibernate valide las entidades. /actuator/health devuelve el estado del servicio; el health check de correo está deshabilitado, por lo que UP no confirma envío de mensajes. Para registro público se necesita un buzón SMTP local o un proveedor de correo configurado.

8. BASE DE DATOS

8.1. Modelo de persistencia y relaciones

La base de control contiene usuarios, roles, planes, suscripciones, instancias, plantillas, incidentes, comentarios, notificaciones, métricas, pagos y facturas. La propiedad de los recursos se representa con claves foráneas. Cada instancia puede agregar un esquema de datos agrícolas cuyo nombre se guarda en instances.db_schema. Esta separación lógica comparte la base y la conexión JDBC del backend; no equivale por sí sola a una base o servidor independiente.



Las relaciones adicionales con users identifican al autor del comentario, al propietario del incidente y a los responsables de asignación y resolución. agrocloud_plantillas no tiene clave foránea hacia instances: el servicio busca la plantilla por su nombre. notifications.resource_id tampoco declara una clave foránea a un recurso concreto.

8.2. Migraciones y estado final del esquema

Flyway aplica las migraciones pendientes en el arranque. La última es V18__instance_db_schema.sql, que agrega db_schema y su índice único. V2 reemplaza users.role por role_code y su relación con roles. V17 agrega verificación de correo, recuperación de contraseña y auth_version. Hibernate usa ddl-auto: validate, por lo que las migraciones deben completar el esquema antes de validar. No modificar scripts ya aplicados: agregar una nueva versión para cambios posteriores.

Versión	Archivo y Finalidad
V1	V1__create_users.sql
V2	V2__create_roles.sql
V3	V3__create_service_plans_subscriptions_and_instances.sql
V4	V4__create_agrocloud_plantillas.sql
V5	V5__create_incidents.sql
V6	V6__plan_active.sql
V7	V7__incident_assignment_and_closure.sql
V8	V8__incident_public_number.sql
V9	V9__incident_resolution_message.sql
V10	V10__incident_notifications.sql
V11	V11__incident_comments.sql
V12	V12__incident_resolution_actor.sql
V13	V13__create_payments_and_invoices.sql
V14	V14__create_instance_metrics.sql
V15	V15__comment_target_role.sql
V16	V16__sync_instances_in_revision.sql
V17	V17__email_verification_and_password_reset.sql
V18	V18__instance_db_schema.sql

8.3. Diccionario de datos de la base de control

El diccionario siguiente representa el resultado acumulado de V1 a V18. PK identifica clave primaria; FK, clave foránea; NOT NULL, dato obligatorio. Los campos sin NOT NULL admiten nulos, salvo restricciones implícitas de una PK. Las referencias y reglas de borrado se resumen después de las tablas.

8.3.1. Users

Campo	Tipo y Restricciones
id	UUID PRIMARY KEY
organization_name	VARCHAR(150) NOT NULL
contact_name	VARCHAR(150)
email	VARCHAR(254) NOT NULL
phone	VARCHAR(30)
password_hash	VARCHAR(100) NOT NULL
status	VARCHAR(20) NOT NULL
created_at	TIMESTAMP WITH TIME ZONE NOT NULL
updated_at	TIMESTAMP WITH TIME ZONE NOT NULL
role_code	VARCHAR(20) NOT NULL; FK roles(code)
email_verified	BOOLEAN NOT NULL DEFAULT TRUE
auth_version	BIGINT NOT NULL DEFAULT 0
verification_token_hash	VARCHAR(64)
verification_expires_at	TIMESTAMP WITH TIME ZONE
verification_sent_at	TIMESTAMP WITH TIME ZONE
reset_token_hash	VARCHAR(64)

8.3.2. Roles

Campo	Tipo y Restricciones
code	VARCHAR(20) PRIMARY KEY

8.3.3. Service_plans

Campo	Tipo y restricciones
id	VARCHAR(50) PRIMARY KEY
name	VARCHAR(100) NOT NULL
storage_gb	DOUBLE PRECISION NOT NULL
monthly_price	NUMERIC(10, 2) NOT NULL
description	TEXT
max_instances	INTEGER NOT NULL
is_popular	BOOLEAN NOT NULL DEFAULT FALSE
is_active	BOOLEAN NOT NULL DEFAULT TRUE

8.3.4. Subscriptions

Campo	Tipo y restricciones
id	UUID PRIMARY KEY
user_id	UUID NOT NULL
plan_id	VARCHAR(50) NOT NULL
status	VARCHAR(20) NOT NULL CHECK (status IN ('pending_payment', 'active', 'suspended', 'cancelled'))
start_date	DATE NOT NULL
next_billing_date	DATE
amount	NUMERIC(10, 2) NOT NULL
created_at	TIMESTAMP WITH TIME ZONE NOT NULL

updated_at	TIMESTAMP WITH TIME ZONE NOT NULL

8.3.5. Instances

Campo	**Tipo y restricciones**
id	UUID PRIMARY KEY
name	VARCHAR(100) NOT NULL
template	VARCHAR(100)
version	VARCHAR(50) NOT NULL DEFAULT 'PostgreSQL 16.2'
status	VARCHAR(20) NOT NULL CHECK (status IN ('active', 'revision', 'suspended', 'terminated'))
region	VARCHAR(100) NOT NULL DEFAULT 'us-east-1 (Virginia)'
host	VARCHAR(150) NOT NULL
port	INTEGER NOT NULL DEFAULT 5432
database_name	VARCHAR(100) NOT NULL
db_user	VARCHAR(100) NOT NULL
encrypted_password	VARCHAR(255) NOT NULL
used_storage_gb	DOUBLE PRECISION NOT NULL DEFAULT 0.5
total_storage_gb	DOUBLE PRECISION NOT NULL DEFAULT 10.0
cpu_usage_percentage	DOUBLE PRECISION NOT NULL DEFAULT 15.0
memory_usage_percentage	DOUBLE PRECISION NOT NULL DEFAULT 30.0
uptime	VARCHAR(20) NOT NULL DEFAULT '99.9%'

owner_id	UUID NOT NULL
created_at	TIMESTAMP WITH TIME ZONE NOT NULL
updated_at	TIMESTAMP WITH TIME ZONE NOT NULL
db_schema	VARCHAR(100)

8.3.6. Agrocloud_plantillas

Campo	**Tipo y restricciones**
id	VARCHAR(100) PRIMARY KEY
nombre	VARCHAR(255) NOT NULL
descripcion	TEXT
tablas	INTEGER
version	VARCHAR(50)
instancias	INTEGER
estado	VARCHAR(50)
schema_tables	TEXT
created_at	TIMESTAMP WITHOUT TIME ZONE
updated_at	TIMESTAMP WITHOUT TIME ZONE

8.3.7. Incidents

Campo	**Tipo y restricciones**
id	UUID PRIMARY KEY
owner_id	UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE
instance_id	UUID NOT NULL REFERENCES instances(id) ON DELETE CASCADE
subject	VARCHAR(200) NOT NULL
category	VARCHAR(80) NOT NULL

problem	TEXT NOT NULL
priority	VARCHAR(10) NOT NULL CHECK (priority IN ('ALTA', 'MEDIA', 'BAJA'))
status	VARCHAR(20) NOT NULL; ABIERTA, EN_REVISION, RESUELTA, CERRADA
diagnostic_guide	TEXT
created_at	TIMESTAMP WITH TIME ZONE NOT NULL
updated_at	TIMESTAMP WITH TIME ZONE NOT NULL
assigned_support_id	UUID REFERENCES users(id) ON DELETE SET NULL
ticket_number	BIGINT NOT NULL UNIQUE; secuencia incident_ticket_number_seq
resolution_message	VARCHAR(2000)
resolved_by_id	UUID REFERENCES users(id)
resolved_at	TIMESTAMP WITH TIME ZONE

8.3.8. Notifications

Campo	**Tipo y restricciones**
id	UUID PRIMARY KEY
recipient_id	UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE
type	VARCHAR(40) NOT NULL
title	VARCHAR(200) NOT NULL
message	TEXT NOT NULL
resource_id	UUID
read_at	TIMESTAMP WITH TIME ZONE
created_at	TIMESTAMP WITH TIME ZONE NOT NULL

8.3.9. Indicent_comments

Campo	Tipo y restricciones
id	UUID PRIMARY KEY
incident_id	UUID NOT NULL REFERENCES incidents(id) ON DELETE CASCADE
author_id	UUID NOT NULL REFERENCES users(id)
body	VARCHAR(2000) NOT NULL
created_at	TIMESTAMP WITH TIME ZONE NOT NULL
target_role	VARCHAR(20) DEFAULT 'CLIENTE'

8.3.10. Payments

Campo	Tipo y restricciones
id	UUID PRIMARY KEY
subscription_id	UUID NOT NULL
amount	NUMERIC(10, 2) NOT NULL
payment_date	TIMESTAMP WITH TIME ZONE NOT NULL
status	VARCHAR(20) NOT NULL CHECK (status IN ('COMPLETED', 'PENDING', 'FAILED', 'REJECTED'))
reference	VARCHAR(100) NOT NULL
payment_method	VARCHAR(50) NOT NULL DEFAULT 'TARJETA'
concept	VARCHAR(255)
created_at	TIMESTAMP WITH TIME ZONE NOT NULL

updated_at	TIMESTAMP WITH TIME ZONE NOT NULL

8.3.11. Invoices

Campo	**Tipo y restricciones**
id	UUID PRIMARY KEY
payment_id	UUID NOT NULL UNIQUE
invoice_number	VARCHAR(50) NOT NULL UNIQUE
issue_date	TIMESTAMP WITH TIME ZONE NOT NULL
subtotal	NUMERIC(10, 2) NOT NULL
tax	NUMERIC(10, 2) NOT NULL
total	NUMERIC(10, 2) NOT NULL
customer_name	VARCHAR(150) NOT NULL
customer_email	VARCHAR(150) NOT NULL
concept	VARCHAR(255) NOT NULL
created_at	TIMESTAMP WITH TIME ZONE NOT NULL

8.3.12. Instance_metrics

Campo	**Tipo y restricciones**
id	UUID PRIMARY KEY
instance_id	UUID NOT NULL
used_storage_gb	DOUBLE PRECISION NOT NULL
total_storage_gb	DOUBLE PRECISION NOT NULL
cpu_usage_percentage	DOUBLE PRECISION NOT NULL
memory_usage_percentage	DOUBLE PRECISION NOT NULL
status	VARCHAR(20) NOT NULL

recorded_at	TIMESTAMP WITH TIME ZONE NOT NULL

8.4. Integridad referencial e índices

roles se relaciona con users mediante role_code. subscriptions referencia users con ON DELETE CASCADE y service_plans sin cascada. instances referencia al propietario con cascada; incidents referencia al propietario y a la instancia con cascada. assigned_support_id usa ON DELETE SET NULL; resolved_by_id referencia users. Los comentarios desaparecen al eliminar el incidente, pero su autor conserva una referencia sin cascada. payments depende de subscriptions; invoices depende de payments y exige payment_id e invoice_number únicos. notifications e instance_metrics se eliminan en cascada con sus respectivos padres.

Los índices cubren rol y estado de usuarios; usuario y estado de suscripción; propietario y estado de instancia; propietario, instancia y asignación de incidente; fechas de comentarios y notificaciones; pagos por suscripción y métricas por instancia y fecha. users.email es único, al igual que los hashes de verificación y restablecimiento. instances.db_schema tiene un índice único. Las restricciones SQL no incluyen una exclusividad de suscripción activa por usuario: ese comportamiento se resuelve en el servicio.

8.5. Esquemas agrícolas y tablas personalizadas

InstanceSchemaProvisioningService construye el nombre con sanitizeSchemaName y el prefijo inst_. InstanceServiceImpl combina el nombre de la instancia con seis caracteres de UUID; el saneamiento limita el componente a 40 caracteres. provisionSchema ejecuta CREATE SCHEMA IF NOT EXISTS y el DDL y los datos de ejemplo definidos en getStatementsForTemplate. Estos esquemas no se crean mediante nuevas versiones Flyway por cada cliente.

El catálogo de plantillas contiene conteos y nombres descriptivos que no deben tomarse como prueba de todas las tablas aprovisionadas. La tabla anterior enumera únicamente los

CREATE TABLE presentes en el servicio de esta revisión. El nombre de la plantilla determina la rama de DDL para riego, plagas, inventario o cultivos.

Para tablas personalizadas se aceptan VARCHAR, TEXT, INTEGER, BIGINT, NUMERIC, BOOLEAN, DATE, TIMESTAMP, SERIAL y BIGSERIAL. VARCHAR admite longitud entre 1 y 1000 o usa 100; NUMERIC se crea como NUMERIC(12,2). Si no se declara una PK, el servicio agrega id SERIAL PRIMARY KEY. El patrón del nombre de tabla es ^[a-zA-Z][a-zA-Z0-9_]{1,40}$ y el de columna ^[a-zA-Z][a-zA-Z0-9_]{0,40}$. La consulta de filas limita el resultado entre 1 y 200; por defecto la API solicita 50.

9. API REST

9.1. Convenciones y autenticación

La base local es http://localhost:8080/api/v1. Los cuerpos utilizan JSON y los identificadores de usuarios, suscripciones, instancias, pagos e incidentes son UUID; los identificadores de planes y plantillas son cadenas. Las rutas protegidas requieren Authorization: Bearer <JWT>. Los roles del dominio son ADMINISTRADOR, SOPORTE y CLIENTE; Spring agrega ROLE_ para las autoridades.

Son públicos los POST de registro, inicio de sesión, verificación, reenvío de verificación y recuperación/restablecimiento de contraseña, el GET exacto /api/v1/planes y /actuator/health. GET /planes/{id} y POST /planes/{id}/cotizar requieren autenticación. Las rutas /usuarios y sus descendientes están limitadas a Administrador. No se encontró dependencia springdoc ni configuración Swagger/OpenAPI en pom.xml; el catálogo se deriva de los controladores reales.

9.2. Catálogo de operaciones

Cada tabla conserva los nombres de controlador, método y DTO del código. «Autenticado» indica la regla de la cadena de seguridad; las comprobaciones por propietario se detallan en las secciones de negocio. Los tipos de retorno indican la estructura de la respuesta, no datos obtenidos de una ejecución de prueba. Las rutas incluyen el prefijo /api/v1.

9.2.1. AdminUserController

Método y Ruta	Entrada	Respuesta y Acceso
GET /api/v1/usuarios	Sin cuerpo JSON	200 · List<UserResponse> Administrador
GET /api/v1/usuarios/{id}	Sin cuerpo JSON	200 · UserResponse Administrador

POST /api/v1/usuarios	JSON AdminCreateUserRequest	201 · UserResponse Administrador
PUT /api/v1/usuarios/{id}	JSON AdminUpdateUserRequest	200 · UserResponse Administrador

9.2.2. AuthController

Método y Ruta	Entrada	Respuesta y Acceso
POST /api/v1/auth/register	JSON RegisterRequest	201 · MessageResponse Público
POST /api/v1/auth/login	JSON LoginRequest	200 · AuthResponse Público
GET /api/v1/auth/me	Sin cuerpo JSON	200 · UserResponse Autenticado
POST /api/v1/auth/verify-email	JSON EmailTokenRequest	200 · AuthResponse Público
POST /api/v1/auth/resend-verification	JSON EmailRequest	200 · MessageResponse Público
POST /api/v1/auth/forgot-password	JSON EmailRequest	200 · MessageResponse Público
POST /api/v1/auth/reset-password	JSON ResetPasswordRequest	200 · MessageResponse Público

9.2.3. IncidentController

Método y Ruta	Entrada	Respuesta y Acceso
GET /api/v1/incidencias	Sin cuerpo JSON	200 · List<IncidentResponse> Autenticado
GET /api/v1/incidencias/actividad	Sin cuerpo JSON	200 · List<IncidentResponse> Autenticado

GET /api/v1/incidencias/cliente/{id}	Sin cuerpo JSON	200 · List<IncidentResponse> Autenticado
GET /api/v1/incidencias/agente/{id}	Sin cuerpo JSON	200 · List<IncidentResponse> Autenticado
POST /api/v1/incidencias	JSON IncidentRequest	201 · IncidentResponse Autenticado
PATCH /api/v1/incidencias/{id}/asignacion	JSON AssignmentRequest	200 · IncidentResponse Autenticado
PATCH /api/v1/incidencias/{id}/estado	JSON StatusRequest	200 · IncidentResponse Autenticado
PATCH /api/v1/incidencias/{id}	JSON IncidentUpdateRequest	200 · IncidentResponse Autenticado
GET /api/v1/incidencias/{id}/comentarios	Sin cuerpo JSON	200 · List<IncidentCommentResponse> Autenticado
POST /api/v1/incidencias/{id}/comentarios	JSON CommentRequest	201 · IncidentCommentResponse Autenticado
PUT /api/v1/incidencias/comentarios/{commentId}	JSON CommentRequest	200 · IncidentCommentResponse Autenticado
DELETE /api/v1/incidencias/comentarios/{commentId}	Sin cuerpo JSON	204 · Void Autenticado

9.2.4. InstanceController

Método y Ruta	Entrada	Respuesta y Acceso
GET /api/v1/instancias	Sin cuerpo JSON	200 · List<InstanceResponse> Autenticado
GET /api/v1/instancias/usuario/{userId}	Sin cuerpo JSON	200 · List<InstanceResponse> Autenticado
GET /api/v1/instancias/{id}	Sin cuerpo JSON	200 · InstanceResponse Autenticado
POST /api/v1/instancias	JSON CreateInstanceRequest	201 · InstanceResponse Autenticado
POST /api/v1/instancias/{id}/restart	Sin cuerpo JSON	200 · InstanceResponse Autenticado
PATCH /api/v1/instancias/{id}/status	status: InstanceStatus	200 · InstanceResponse Administrador o Soporte
DELETE /api/v1/instancias/{id}	Sin cuerpo JSON	204 · Void Autenticado
GET /api/v1/instancias/{id}/credenciales	Sin cuerpo JSON	200 · InstanceCredentialResponse Autenticado
GET /api/v1/instancias/{id}/metricas	Sin cuerpo JSON	200 · InstanceMonitoringResponse Autenticado
GET /api/v1/instancias/{id}/tablas	Sin cuerpo JSON	200 · List<Map<String, Object>> Autenticado
GET /api/v1/instancias/{id}/tablas/{tableName}/datos	limit: int (defaultValue = "50")	200 · List<Map<String, Object>> Autenticado

GET /api/v1/instancias/{id}/tablas/{tableName}/columnas	Sin cuerpo JSON	200 · List<Map<String, Object>> Autenticado
POST /api/v1/instancias/{id}/tablas/{tableName}/filas	JSON Map<String, Object>	201 · Map<String, Object> Autenticado
POST /api/v1/instancias/{id}/tablas	JSON CreateCustomTableRequest	201 · Map<String, String> Autenticado

9.2.5. NotificationController

Método y Ruta	Entrada	Respuesta y Acceso
GET /api/v1/notificaciones	Sin cuerpo JSON	200 · List<NotificationResponse> Autenticado
PATCH /api/v1/notificaciones/{id}/leida	Sin cuerpo JSON	200 · NotificationResponse Autenticado
GET /api/v1/notificaciones/stream	Sin cuerpo JSON	200 · SseEmitter Autenticado

9.2.6. InvoiceController

Método y Ruta	Entrada	Respuesta y Acceso
GET /api/v1/facturas/{pagoId}	Sin cuerpo JSON	200 · InvoiceResponse Autenticado
GET /api/v1/facturas/pago/{pagoId}	Sin cuerpo JSON	200 · InvoiceResponse Autenticado

9.2.7. PaymentController

Método y Ruta	Entrada	Respuesta y Acceso
POST /api/v1/pagos	JSON CreatePaymentRequest	201 · PaymentResponse Autenticado
GET /api/v1/pagos/suscripcion/{subscriptionId}	Sin cuerpo JSON	200 · List<PaymentResponse> Autenticado
GET /api/v1/pagos/me	Sin cuerpo JSON	200 · List<PaymentResponse> Autenticado
GET /api/v1/pagos	Sin cuerpo JSON	200 · List<PaymentResponse> Administrador o Soporte

9.2.8. PlanController

Método y Ruta	Entrada	Respuesta y Acceso
GET /api/v1/planes	Sin cuerpo JSON	200 · List<PlanResponse> Público
GET /api/v1/planes/{id}	Sin cuerpo JSON	200 · PlanResponse Autenticado
POST /api/v1/planes/{id}/cotizar	JSON CotizarPlanRequest	200 · CotizacionPlanResponse Autenticado
POST /api/v1/planes	JSON UpsertPlanRequest	201 · PlanResponse Administrador
PUT /api/v1/planes/{id}	JSON UpsertPlanRequest	200 · PlanResponse Administrador

DELETE /api/v1/planes/{id}	Sin cuerpo JSON	204 · Void Administrador

9.2.9. SubscriptionController

Método y Ruta	Entrada	Respuesta y Acceso
POST /api/v1/suscripciones	JSON CreateSubscriptionRequest	201 · SubscriptionResponse Autenticado
GET /api/v1/suscripciones/activa	Sin cuerpo JSON	200 · SubscriptionResponse Autenticado
GET /api/v1/suscripciones/me	Sin cuerpo JSON	200 · List<SubscriptionResponse> Autenticado
GET /api/v1/suscripciones/usuario/{userId}	Sin cuerpo JSON	200 · List<SubscriptionResponse> Autenticado
GET /api/v1/suscripciones	Sin cuerpo JSON	200 · List<SubscriptionResponse> Administrador o Soporte
PATCH /api/v1/suscripciones/{id}/status	status: SubscriptionStatus	200 · SubscriptionResponse Administrador
POST /api/v1/suscripciones/{id}/renovar	Sin cuerpo JSON	200 · SubscriptionResponse Autenticado
POST /api/v1/suscripciones/renovar	id: UUID	200 · SubscriptionResponse Autenticado

POST /api/v1/suscripciones/{id}/suspender	Sin cuerpo JSON	200 · SubscriptionResponse Autenticado
POST /api/v1/suscripciones/suspender	id: UUID	200 · SubscriptionResponse Autenticado
POST /api/v1/suscripciones/{id}/cancelar	Sin cuerpo JSON	200 · SubscriptionResponse Autenticado

9.2.10. TemplateController

Método y Ruta	Entrada	Respuesta y Acceso
GET /api/v1/plantillas	Sin cuerpo JSON	200 · List<TemplateDto> Autenticado
POST /api/v1/plantillas	JSON CreateTemplateDto	201 · TemplateDto Administrador
DELETE /api/v1/plantillas/{id}	Sin cuerpo JSON	204 · Void Administrador
PUT /api/v1/plantillas/{id}	JSON CreateTemplateDto	200 · TemplateDto Administrador

9.3. Contratos de entrada y salida

Los siguientes contratos muestran los componentes de los records Java expuestos por la API. @Valid y las anotaciones de cada solicitud se aplican en los controladores que las declaran. Las fechas se serializan desde Instant o LocalDate; BigDecimal representa importes. Una lista o mapa de tablas utiliza las claves consultadas en PostgreSQL y las claves nombre, esquema y totalFilas para el listado de tablas.

Contrato	Campos JSON y tipos Java
AdminCreateUserRequest	String organizationName, String contactName, String email, String phone, String password, Role role
AdminUpdateUserRequest	String organizationName, String contactName, String email, String phone, String password, Role role, AccountStatus status
AuthResponse	String token, String tokenType, long expiresIn, UserResponse user
EmailRequest	String email
EmailTokenRequest	String token
LoginRequest	String email, String password
MessageResponse	String message
RegisterRequest	String organizationName, String contactName, String email, String phone, String password
ResetPasswordRequest	String token, String password
UserResponse	UUID id, String organizationName, String contactName, String email, String phone, Role role, AccountStatus status, Instant createdAt
IncidentCommentResponse	UUID id, UUID autorId, String autor, Role rolAutor, String texto, String targetRole, Instant fecha
IncidentRequest	UUID instanciaId, String asunto, String categoria, String problema, String prioridad
IncidentResponse	UUID id, String codigo, String cliente, UUID instanciaId, String instanciaNombre, String plantilla, String asunto, String categoria, String problema, String prioridad, String

	estado, Instant fecha, Instant actualizado, String guiaDiagnostico, UUID agenteId, String mensajeResolucion, UUID resueltoPorId, Instant fechaResolucion
CreateCustomTableRequest	String tableName, List<ColumnDefinition> columns
CreateInstanceRequest	String name, String template
InstanceCredentialResponse	UUID instanceId, String instanceName, String host, Integer port, String databaseName, String dbSchema, String dbUser, String password, String region, String versionEngine
InstanceMetricResponse	UUID id, UUID instanceId, Double usedStorageGb, Double totalStorageGb, Double cpuUsagePercentage, Double memoryUsagePercentage, InstanceStatus status, Instant recordedAt
InstanceMonitoringResponse	UUID instanceId, String instanceName, InstanceStatus currentStatus, Double usedStorageGb, Double totalStorageGb, Double storagePercentage, Double cpuPercentage, Double memoryPercentage, String uptime, List<InstanceMetricResponse> metricHistory
InstanceResponse	UUID id, String name, String template, String clientOrganization, UUID ownerUserId, String version, InstanceStatus status, String dbEngineVersion, String uptime, Double cpuUsagePercentage, Double memoryUsagePercentage, Double usedStorageGb, Double totalStorageGb, String region, String host, Integer port, String

	databaseName, String dbSchema, String dbUser, LocalDate createdAt
NotificationResponse	UUID id, String tipo, String titulo, String mensaje, UUID recursoId, Instant fecha, boolean leida
CreatePaymentRequest	UUID subscriptionId, BigDecimal monto, String referencia, String metodoPago, String concepto
InvoiceResponse	UUID id, UUID pagoId, UUID suscripcionId, String numeroFactura, Instant fechaEmision, String clienteNombre, String clienteEmail, String concepto, BigDecimal subtotal, BigDecimal impuesto, BigDecimal total, String metodoPago, String referenciaPago, String estado
PaymentResponse	UUID id, UUID suscripcionId, String planNombre, BigDecimal monto, Instant fecha, PaymentStatus estado, String referencia, String metodoPago, String concepto
CotizacionPlanResponse	String planId, String descripcion, BigDecimal precioMensual, Double almacenamientoGb, Integer instanciasPermitidas, List<String> complementos
CotizarPlanRequest	List<Complemento> complementos
PlanResponse	String id, String name, Double storageGb, BigDecimal monthlyPrice, String description, Integer maxInstances, Boolean popular, Boolean active
UpsertPlanRequest	String nombre, Double almacenamientoGb, BigDecimal precioMensual, String

	descripcion, Integer instanciasPermitidas, Boolean popular, Boolean activo
CreateSubscriptionRequest	String planId, List<Complemento> complementos
SubscriptionResponse	UUID id, String planId, String planName, UUID userId, SubscriptionStatus status, LocalDate startDate, LocalDate nextBillingDate, BigDecimal amount
CreateTemplateDto	String nombre, String descripcion, String version, String estado, List< String> schema
TemplateDto	String id, String nombre, String descripcion, Integer tablas, String version, Integer instancias, String estado, String actualizada, List<String> schema

Contratos anidados adicionales: ColumnDefinition contiene name, type, length, nullable, primaryKey y defaultValue; AssignmentRequest contiene agenteId (UUID obligatorio); StatusRequest contiene estado (obligatorio) y mensajeResolucion (máximo 2000); IncidentUpdateRequest contiene estado, guiaDiagnostico y mensajeResolucion; CommentRequest contiene texto (obligatorio, máximo 2000) y destinatario. Los identificadores entre llaves en las rutas son parámetros de trayectoria y deben sustituirse antes de enviar la solicitud.

9.4. Respuestas de error

GlobalExceptionHandler construye ApiError con timestamp, status, error, message, path y fieldErrors. Los mensajes incluyen «La solicitud contiene datos no válidos», «El cuerpo JSON no es válido» y «Un parámetro de la URL no es válido». La ausencia de suscripción activa genera 404. Un exceso de cuota de instancias genera 400. Las respuestas de Spring Security pueden seguir un formato distinto al manejador funcional.

HTTP	Situación y representación
400	Validación de DTO, JSON mal formado, parámetro incompatible, IllegalArgumentException o InstanceLimitExceededException.
401	Credenciales incorrectas o solicitud sin autenticación válida.
403	Cuenta inactiva o acceso no autorizado por rol o propietario. Algunos controladores devuelven la respuesta sin cuerpo.
404	Usuario, instancia, plan o suscripción inexistente.
409	Correo duplicado, conflicto de integridad o IllegalStateException.
500	Fallo no traducido por un manejador específico. No asumir que todas las excepciones de aprovisionamiento tienen una respuesta funcional controlada.

10. MÓDULOS Y REGLAS DE NEGOCIO

10.1. Usuarios y cuentas

AuthController coordina RegistrationService, LoginService y EmailAccountService. El registro público crea una cuenta de cliente pendiente de verificación. El correo se normaliza mediante EmailAddress; UserRepository comprueba duplicados. Las contraseñas se procesan con BCryptPasswordEncoder(12). El registro y la administración exigen entre 8 y 72 caracteres; no hay una regla general que obligue a combinar letras, números y símbolos.

AdminUserController y AdminUserService permiten listar, consultar, crear y actualizar usuarios. No hay endpoint DELETE para usuarios. Una cuenta creada por administrador queda ACTIVO. En la edición, un administrador no puede cambiar su propio rol ni suspender su cuenta. Al modificar correo o contraseña se incrementa auth_version y se limpian los tokens de recuperación y verificación. UserResponse omite el hash de contraseña y los tokens.

Tablas: users y roles. Los DTO AdminCreateUserRequest y AdminUpdateUserRequest validan organización, correo y teléfono, además de rol y estado donde corresponda. La persistencia usa la relación role_code con roles. AuthResponseFactory y JwtService construyen la respuesta de acceso. Las pruebas de referencia incluyen RegistrationServiceTest, LoginServiceTest, EmailAccountServiceTest y AdminUserServiceTest; su existencia no implica que se hayan ejecutado durante la elaboración del manual.

10.2. Planes y complementos

PlanController y PlanServiceImpl administran service_plans. El listado es público, mientras crear, editar y eliminar requieren Administrador. PlanDataInitializer contiene los valores iniciales del catálogo. Debe distinguirse el precio y límites del plan base del resultado de una cotización con complementos.

PlanContratable define el contrato común; PlanBase adapta PlanEntity; DecoradorPlan delega el comportamiento y los decoradores concretos agregan capacidades o precio. PlanCotizador aplica cada Complemento en el orden recibido. Los códigos admitidos son ALMACENAMIENTO_EXTRA, INSTANCIA_EXTRA, RESPALDO_DIARIO y SOPORTE_PRIORITARIO. La cotización devuelve CotizacionPlanResponse; los complementos de respaldo y soporte representan una oferta en la cotización y no prueban un proceso automático de respaldo o una operación de soporte implementada.

PlanDecoratorTest cubre la composición del plan. La lista de complementos puede estar ausente y se usa el plan base; un elemento nulo se rechaza. El algoritmo recorre la lista sin eliminar duplicados, por lo que repetir un complemento lo aplica más de una vez.

10.3. Suscripciones y contratación

d SubscriptionController recibe CreateSubscriptionRequest y delega en SubscriptionServiceImpl. La creación obtiene al usuario autenticado, verifica que el plan esté activo y cotiza sus complementos. Si el número actual de instancias supera el máximo cotizado, rechaza el cambio. La suscripción activa anterior se marca cancelled, se crea una nueva active con fecha de siguiente cobro un mes después y se registra un pago automático. Los datos se guardan en subscriptions, payments e invoices mediante los servicios de pago.

renovarSubscription rechaza las suscripciones canceladas, activa la suscripción, adelanta un mes la fecha siguiente y registra un pago automático. suspenderSubscription rechaza una suscripción ya suspendida o cancelada; cancelarSubscription rechaza una que ya esté cancelada. Los estados persistidos son pending_payment, active, suspended y cancelled.

Limitación de la revisión: los complementos no se guardan como filas o capacidades propias de la suscripción. amount conserva el precio cotizado, pero la creación de instancias consulta activeSub.getPlan().getMaxInstances() y getStorageGb() del plan base. Por ello, una cuota adicional cotizada no queda plenamente aplicada al aprovisionamiento posterior. Suspender una

suscripción cambia su estado; no hay propagación en ese método que suspenda automáticamente todas sus instancias.

La lista global requiere Administrador o Soporte; la actualización PATCH de estado requiere Administrador. Las consultas por usuario permiten al propietario y a Administrador/Soporte. Los seis alias POST de renovar, suspender y cancelar no incorporan @PreAuthorize ni una comprobación de propietario en el servicio: en esta versión solo se exige autenticación. La restricción por botones del frontend no debe documentarse como una garantía de la API.

10.4. Instancias y aprovisionamiento de esquemas

InstanceController, InstanceServiceImpl, InstanceRepository e InstanceSchemaProvisioningService gestionan instances y sus esquemas. Crear una instancia requiere una suscripción activa, cuota libre y una plantilla existente en estado Activa. El conteo de cuota utiliza countByOwnerId y no excluye las instancias suspendidas o terminadas. El espacio total asignado se obtiene del plan base.

El flujo de creación verifica el usuario y suscripción, genera datos de conexión y una contraseña cifrada con EncryptionService, determina el esquema, crea las tablas y los datos de ejemplo y guarda la instancia activa. NotificationService registra la notificación al propietario. El aprovisionamiento SQL utiliza el datasource compartido. La cadena de conexión externa puede resolverse a partir del datasource remoto o de PUBLIC_DB_*; su esquema debe coincidir con db_schema.

Listar y consultar recursos distingue Administrador/Soporte del propietario. El cambio PATCH /status solo permite Administrador o Soporte. Los estados reales son active, revision, suspended y terminated. restartInstance cambia el estado a active, guarda una métrica y emite una notificación; no ejecuta un reinicio del servidor PostgreSQL. La eliminación borra las métricas, solicita DROP SCHEMA IF EXISTS ... CASCADE y elimina la entidad. Esta operación puede

destruir todas las tablas del esquema. dropSchema registra y captura sus errores; no debe asumirse limpieza física confirmada únicamente por una respuesta HTTP exitosa.

Las métricas combinan almacenamiento calculado con pg_total_relation_size y valores de CPU/RAM generados por el servicio; cuando no hay historial se crean muestras. No constituyen telemetría integral del servidor. instance_metrics persiste las muestras. InstanceNotificationTest verifica interacciones de notificación y sirve como referencia del flujo.

10.5. Explorador de tablas y datos

InstanceDataExplorerModal del frontend consume los métodos de instancesApi.ts para listar tablas, consultar columnas, leer filas, insertar un registro y crear tablas. El backend comprueba la instancia y el propietario antes de trabajar con su db_schema. Si no hay esquema, los listados pueden devolver una lista vacía; las operaciones de escritura lanzan un conflicto de estado. Las operaciones disponibles no incluyen edición o eliminación de filas mediante estos endpoints.

getTableRows filtra identificadores y acota limit. insertTableRow omite la columna id, valida los nombres y envía los valores mediante parámetros JDBC. createCustomTable valida los tipos permitidos y construye el DDL. El esquema deriva de la entidad de la instancia, no de un parámetro libre del cliente. Esta protección en la aplicación no demuestra aislamiento por credenciales PostgreSQL: la conexión pública puede utilizar un usuario compartido y requiere revisar privilegios antes de habilitar acceso directo.

10.6. Dependencias con incidentes y facturación

IncidentService relaciona incidentes con el propietario y la instancia, administra asignación y resolución y genera notificaciones. IncidentCommentService mantiene los comentarios y target_role. Los estados son ABIERTA, EN_REVISION, RESUELTA y CERRADA. PaymentService registra pagos y factura para contratación o renovación; se trata de un flujo simulado, sin evidencia de integración de cobro bancario real en estos servicios. Estos módulos

conectan las secciones del presente aporte con seguridad, pruebas y facturación de los otros integrantes.

10.7. Evidencias de implementación y límites de la versión

Evidencia del repositorio	Qué respalda
backend/pom.xml y frontend/package-lock.json	Tecnologías y requisitos reales; prevalecen sobre descripciones desactualizadas del README.
application.yml y docker-compose.e04.yml	Variables, puertos, datasource, Flyway y construcción de servicios.
db/migration/V1 a V18	Esquema de control, restricciones, índices y datos iniciales.
Controladores y DTO de backend/src/main/java	Métodos, rutas, parámetros y contratos del catálogo de la sección 7.
InstanceSchemaProvisioningService	DDL agrícola, creación de tablas personalizadas e inserciones parametrizadas.
PlanCotizador y SubscriptionServiceImpl	Composición de precios, contratación y límites de persistencia de complementos.
ARQUITECTURA_MEJORAS.md	Propuestas de mejora; no se describen como clases o funciones ya incorporadas.

11. FACTURACIÓN, PAGOS Y MÉTRICAS DE RECURSOS

AgroCloud permite consultar el consumo de recursos asociados a las bases de datos, administrar la información de los pagos realizados y acceder a los comprobantes de facturación de las suscripciones. Estas funcionalidades facilitan el seguimiento de los servicios contratados y permiten conocer el estado de los recursos disponibles según las condiciones de cada plan.

La información se presenta mediante paneles de monitoreo, indicadores de consumo y registros de transacciones. Los clientes pueden consultar sus propios datos de facturación y almacenamiento, mientras que los administradores disponen de herramientas para supervisar las operaciones financieras de la plataforma.

11.1. Monitoreo de recursos del sistema

El monitoreo de recursos permite conocer el estado de las instancias de bases de datos y verificar el consumo de almacenamiento en relación con los límites establecidos por el plan contratado. Esta información se encuentra disponible en el Dashboard de AgroCloud, donde se muestran indicadores que facilitan la supervisión de los servicios.

11.1.1. Consulta de métricas de recursos

Para consultar las métricas, el usuario debe iniciar sesión con una cuenta autorizada y acceder a la opción Dashboard desde el menú lateral. En la parte superior se muestra la información de la suscripción activa, incluyendo el nombre del plan y la capacidad de almacenamiento asignada.

El panel presenta la cantidad de instancias activas en comparación con el límite permitido, el almacenamiento utilizado expresado en gigabytes y el estado de disponibilidad de los servicios. También incorpora una representación gráfica del consumo mensual, que permite identificar variaciones en el uso de los recursos.

La consulta periódica de estos indicadores facilita la identificación de posibles limitaciones de capacidad y permite evaluar la necesidad de modificar el plan contratado antes de alcanzar los recursos disponibles.

11.2. Historial de pagos

El historial de pagos permite consultar las transacciones registradas por concepto de contratación o renovación de suscripciones. Cada operación conserva información relacionada con el servicio adquirido, la fecha correspondiente, el monto y el estado del pago.

11.2.1. Consulta de transacciones

Para acceder al historial, el cliente debe seleccionar la opción Pagos desde el menú lateral de AgroCloud. El sistema muestra una tabla con los registros asociados a su cuenta, incluyendo el identificador de la transacción, fecha de emisión, concepto, período de facturación, monto en quetzales, método de pago y estado.

Los estados permiten distinguir las transacciones pagadas, pendientes y rechazadas. Además, los filtros disponibles facilitan la búsqueda de operaciones correspondientes a períodos anteriores.

Esta funcionalidad permite mantener un control organizado de los pagos realizados y consultar la información de facturación sin necesidad de revisar cada suscripción individualmente.

11.3. Consulta de facturas

Las facturas contienen el detalle de los servicios contratados y la información correspondiente a las transacciones registradas. Su consulta permite revisar los conceptos facturados y conservar un respaldo de las operaciones realizadas dentro de AgroCloud.

11.3.1. Visualización y descarga de facturas

Para consultar una factura, el cliente debe ingresar a la sección Pagos, localizar la transacción correspondiente y seleccionar la opción Ver Factura.

AgroCloud muestra una ventana con la información del emisor, los datos del cliente, el servicio contratado y el desglose de los importes, incluyendo el subtotal y los impuestos indicados en el comprobante.

Cuando se requiere conservar una copia, el usuario puede utilizar la opción Imprimir / Guardar PDF, que permite generar un archivo para su almacenamiento o consulta posterior.

11.4. Auditoría de transacciones financieras

La auditoría de transacciones permite a los administradores consultar de manera general los movimientos económicos registrados en AgroCloud. A diferencia del historial individual de los clientes, esta funcionalidad proporciona una visión consolidada de las operaciones relacionadas con las suscripciones.

11.4.1. Supervisión de pagos

El administrador debe acceder a la consola administrativa y seleccionar la opción Auditoría de Pagos. Desde esta sección puede consultar los ingresos registrados por período, las transacciones completadas y los pagos rechazados.

También dispone de filtros por fechas y clientes, lo que facilita la localización de movimientos específicos y la revisión de los registros financieros.

La información obtenida permite identificar el comportamiento de las transacciones, verificar los estados de pago y mantener un seguimiento administrativo de los servicios contratados.

12. ERRORES Y PREGUNTAS FRECUENTES

Durante la utilización de AgroCloud pueden presentarse situaciones relacionadas con el acceso a las cuentas, la disponibilidad de los servicios, los límites de las suscripciones o la conexión con herramientas externas. La identificación de estos problemas permite aplicar las medidas correspondientes y recuperar el funcionamiento esperado.

Los errores pueden originarse por información incorrecta, permisos insuficientes, configuraciones de conexión o restricciones establecidas por el sistema. Por ello, es importante reconocer los mensajes mostrados por la plataforma y comprender las acciones necesarias para resolverlos.

12.1. Diagnóstico y resolución de errores

AgroCloud utiliza mensajes de validación y códigos de respuesta HTTP para informar sobre situaciones que impiden completar determinadas operaciones. Estos mensajes permiten identificar la causa general del problema y orientar al usuario hacia una posible solución.

12.1.1. Consulta de métricas de recursos

Los errores de autenticación ocurren cuando las credenciales ingresadas no coinciden con la información registrada en el sistema. En estos casos, AgroCloud puede mostrar el mensaje Credenciales incorrectas, acompañado de una respuesta HTTP 401.

Para resolverlo, el usuario debe verificar que el correo electrónico y la contraseña estén escritos correctamente, comprobar que no existan espacios adicionales y revisar el uso de mayúsculas y minúsculas.

Cuando no recuerda su contraseña, puede utilizar la opción ¿Olvidaste tu contraseña? para iniciar el proceso de recuperación.

12.1.2. Errores de validación de contraseñas

Durante el registro o actualización de credenciales, AgroCloud verifica que las contraseñas cumplan con los requisitos establecidos. Una contraseña que no respete la longitud permitida puede generar una respuesta HTTP 400.

El sistema establece una longitud de entre 8 y 72 caracteres. Para evitar errores, se recomienda utilizar combinaciones de letras mayúsculas, minúsculas y números, procurando que la contraseña no contenga información personal fácilmente identificable.

12.1.3. Errores de permisos y acceso

Los errores de autorización se producen cuando un usuario intenta ingresar a una funcionalidad que no corresponde a los permisos de su cuenta. AgroCloud aplica restricciones según los roles de cliente, administrador y soporte técnico.

Cuando se intenta realizar una operación no autorizada, el sistema puede responder con el código HTTP 403, indicando que el acceso fue denegado.

Para resolver esta situación, debe verificarse que la cuenta utilizada tenga los permisos correspondientes. Las funcionalidades administrativas y técnicas requieren credenciales asociadas a los roles autorizados.

12.1.4. Límite de instancias alcanzado

Cada plan de suscripción establece una cantidad máxima de bases de datos que pueden mantenerse activas. Cuando un cliente intenta crear una instancia adicional después de alcanzar ese límite, AgroCloud impide completar la operación.

Para continuar, el usuario puede eliminar una instancia que ya no necesite o consultar las opciones disponibles para cambiar a un plan con mayor capacidad.

Esta restricción permite controlar la asignación de recursos y mantener el consumo dentro de las condiciones de la suscripción.

12.1.5. Errores de conectividad externa

Los problemas de conectividad pueden aparecer al intentar acceder a una base de datos desde aplicaciones externas como DBeaver, herramientas de análisis o sistemas desarrollados por el cliente.

Una causa posible es la configuración incorrecta del host, puerto, usuario, contraseña o esquema asignado. Debido a que AgroCloud utiliza esquemas independientes para organizar las bases de datos de sus clientes, también es necesario comprobar que la conexión haga referencia al esquema correspondiente.

El usuario debe revisar los datos proporcionados en la sección de credenciales de la instancia y verificar que coincidan con los parámetros configurados en la aplicación externa.

12.1.6. Expiración del enlace de verificación

Los enlaces enviados para confirmar una cuenta tienen una vigencia limitada de 24 horas. Cuando se intenta utilizar un enlace después de ese período, AgroCloud informa que el token de verificación ha expirado.

Para completar la activación, el usuario debe solicitar un nuevo correo de confirmación desde la pantalla correspondiente y utilizar el enlace actualizado.

12.1.7. Indisponibilidad temporal de instancias

Durante determinadas operaciones de mantenimiento o reinicio, una instancia puede encontrarse temporalmente indisponible. En estos casos, las solicitudes de conexión pueden generar una respuesta de error hasta que el servicio recupere su estado operativo.

Cuando una instancia aparece en estado Reiniciando, se recomienda esperar aproximadamente entre 10 y 15 segundos y verificar nuevamente su disponibilidad desde el panel de administración.

Si el problema continúa, el usuario puede registrar una incidencia para solicitar asistencia técnica.

12.2. Preguntas frecuentes sobre planes y almacenamiento

Las condiciones de las suscripciones determinan la cantidad de almacenamiento y el número de instancias que puede utilizar cada cliente. Conocer estas características permite seleccionar un plan adecuado y administrar los recursos disponibles.

12.2.1. Planes disponibles

AgroCloud dispone de cuatro planes de suscripción orientados a diferentes necesidades de almacenamiento y administración de bases de datos.

El plan Finca tiene un costo de Q25.00 mensuales e incluye 10 GB de almacenamiento y una base de datos. El plan Productor ofrece 50 GB y una base de datos por Q60.00 mensuales.

Para necesidades mayores, Agro Pro proporciona 100 GB y dos bases de datos por Q120.00 mensuales, mientras que Agro Enterprise incluye 250 GB, cinco bases de datos y soporte dedicado por Q250.00 mensuales.

12.2.2. Límite de almacenamiento

El almacenamiento disponible depende de la capacidad contratada. AgroCloud contempla una alerta preventiva cuando el consumo alcanza el 85 % de la cuota asignada.

Al llegar al 100 %, la base de datos entra en modo de solo lectura para evitar nuevas operaciones de escritura mientras se libera espacio o se amplía la capacidad mediante un cambio de plan.

La revisión periódica de las métricas permite anticipar estas situaciones y reducir las interrupciones relacionadas con la falta de almacenamiento.

12.2.3. Consulta de métricas de recursos

El cambio de plan permite modificar los recursos disponibles sin eliminar los registros almacenados en las bases de datos existentes.

Las actualizaciones de suscripción conservan los esquemas y la información registrada, por lo que el cliente puede ampliar su capacidad de almacenamiento o número de instancias sin necesidad de crear nuevamente sus estructuras de datos.

12.3. Preguntas frecuentes sobre conectividad

AgroCloud permite utilizar las bases de datos desde aplicaciones externas compatibles con PostgreSQL. Esta capacidad facilita la integración con herramientas de administración, sistemas agrícolas y aplicaciones desarrolladas por los clientes.

12.3.1. Conexión de aplicaciones externas

Las bases de datos pueden utilizarse desde aplicaciones que admitan conexiones con PostgreSQL, incluyendo programas de escritorio, sistemas web y herramientas destinadas al procesamiento de información agrícola.

Para establecer una conexión es necesario disponer de los parámetros correspondientes y verificar que la red permita la comunicación con el servidor de bases de datos.

La compatibilidad con el protocolo de PostgreSQL facilita la integración con diferentes tecnologías de desarrollo, siempre que se respeten las condiciones de acceso y seguridad configuradas.

12.3.2. Consulta de credenciales de conexión

Los datos de conexión se encuentran disponibles en la sección Instancias de BD. El cliente debe localizar la instancia correspondiente y seleccionar la opción Credenciales.

AgroCloud muestra la información necesaria para configurar el acceso, incluyendo host, puerto, esquema, usuario y contraseña.

Estos datos deben utilizarse únicamente en aplicaciones autorizadas y almacenarse mediante mecanismos seguros para evitar accesos no permitidos.

13. SEGURIDAD PARA USUARIOS Y MESA DE SOPORTE

La seguridad de AgroCloud depende tanto de los mecanismos implementados en la plataforma como del uso adecuado de las cuentas y credenciales por parte de sus usuarios. La protección de las contraseñas, el control de acceso y la administración responsable de las sesiones contribuyen a reducir los riesgos de exposición de información.

Asimismo, la plataforma dispone de herramientas de soporte técnico que permiten registrar problemas, mantener comunicación con el personal responsable y dar seguimiento a las incidencias relacionadas con los servicios contratados.

13.1. Recomendaciones de seguridad

Las recomendaciones de seguridad buscan reducir los riesgos asociados con el uso indebido de credenciales, el acceso no autorizado y la exposición de información almacenada en las bases de datos.

13.1.1. Protección de credenciales de base de datos

Las credenciales de conexión permiten acceder a la información almacenada en las instancias, por lo que deben mantenerse protegidas y utilizarse únicamente en aplicaciones autorizadas.

Se recomienda evitar el envío de contraseñas mediante canales de comunicación no seguros. Cuando se desarrollen aplicaciones externas, las credenciales pueden almacenarse en variables de entorno o archivos de configuración locales que no formen parte del código fuente compartido.

En proyectos que utilizan Git, los archivos que contienen información confidencial deben excluirse del control de versiones mediante .gitignore, evitando su publicación accidental en repositorios.

13.1.2. Seguridad de contraseñas de usuario

Las contraseñas utilizadas para acceder a AgroCloud deben ser difíciles de adivinar y diferentes de las empleadas en otros servicios.

Se recomienda evitar nombres de fincas, fechas de nacimiento, números telefónicos y combinaciones comunes que puedan relacionarse fácilmente con el propietario de la cuenta.

Si existe sospecha de acceso no autorizado, el usuario debe restablecer su contraseña mediante el mecanismo de recuperación disponible. Esta operación permite invalidar las sesiones anteriores y proteger nuevamente el acceso a la cuenta.

13.1.3. Cierre de sesión en equipos compartidos

El uso de AgroCloud desde computadoras compartidas requiere especial atención para evitar que otras personas accedan a información privada.

Al finalizar las actividades, el usuario debe utilizar la opción Cerrar sesión y verificar que la plataforma regrese a la pantalla de autenticación.

También se recomienda evitar guardar contraseñas en navegadores de equipos públicos o pertenecientes a terceros.

13.2. Atención y resolución de incidencias

La atención de incidencias corresponde al personal de soporte técnico y a los administradores autorizados. Su función es revisar las solicitudes recibidas, identificar las causas de los problemas y aplicar las acciones necesarias para resolverlos.

13.2.1. Protección de credenciales de base de datos

El personal autorizado debe ingresar al panel de soporte y seleccionar la opción Cola de Incidentes. Desde esta sección puede consultar los tickets registrados y seleccionar aquellos que se encuentran pendientes de atención.

Al iniciar la revisión, el estado de la incidencia puede modificarse de Abierto a En Progreso, indicando que el problema está siendo analizado.

Durante el diagnóstico, el personal técnico puede solicitar información adicional, revisar los mensajes de error y proporcionar instrucciones para realizar pruebas de conectividad o configuración.

13.2.2. Resolución y cierre de incidencias

Una vez identificado y solucionado el problema, el responsable debe actualizar el estado del ticket a Resuelto.

El registro de resolución requiere una nota técnica que describa la causa identificada y las acciones aplicadas para solucionar el inconveniente. Esta información permite conservar un historial de las medidas realizadas y facilita la atención de problemas similares.

Finalmente, mediante la opción Guardar y Notificar al Cliente, se registra la resolución y se genera una notificación para informar al usuario sobre el resultado de la atención.

Figura 1. *Modelo del dominio completo*

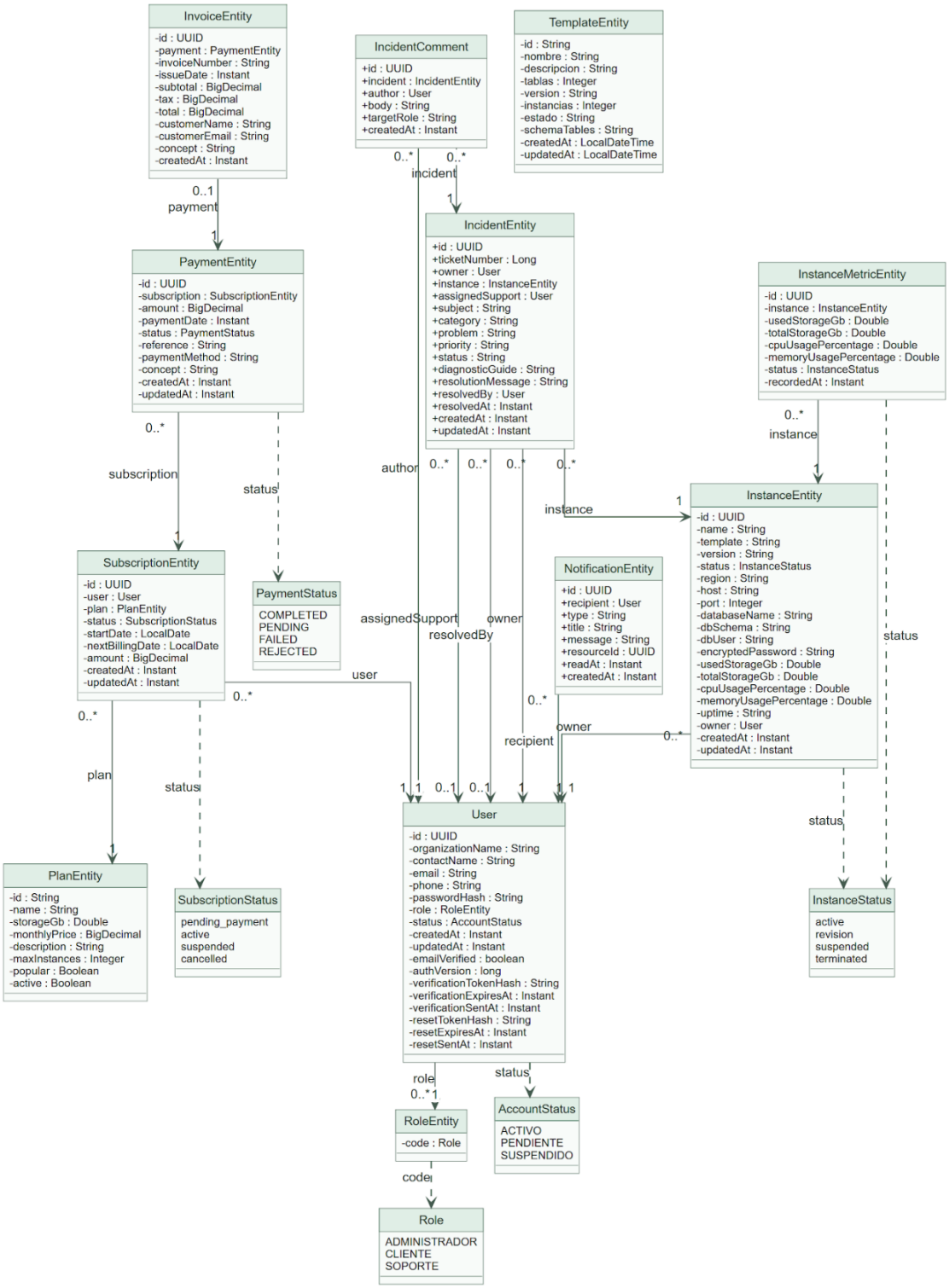


Figura 2. *Clases de servicios y colaboradores*

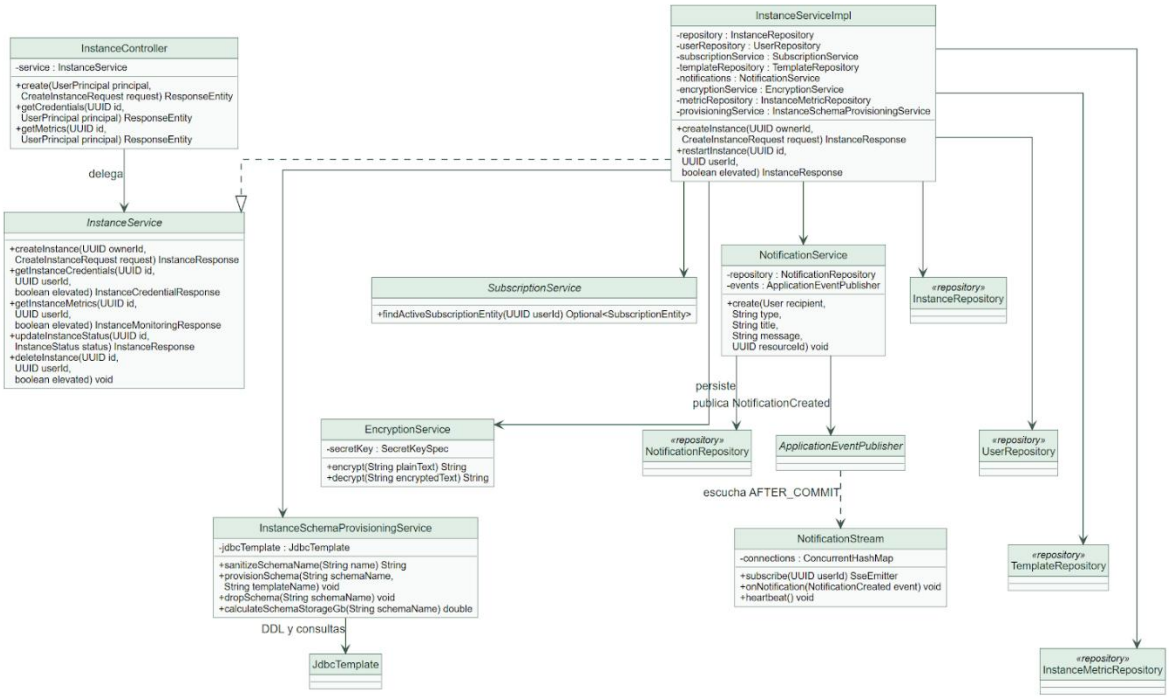


Figura 3. *AgroCloud - Acceso de cliente*

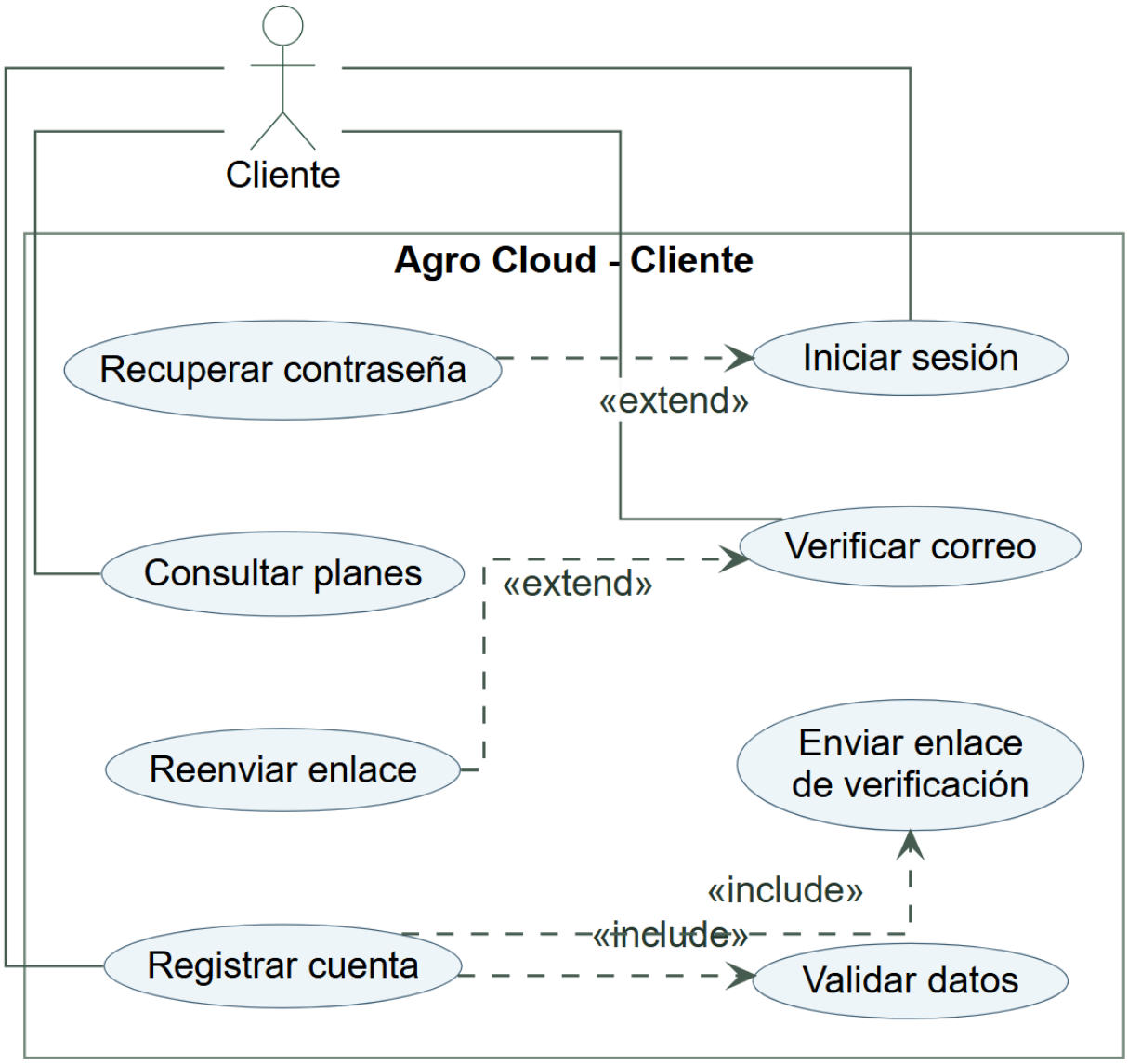


Figura 4. *AgroCloud - Acceso de clientes a servicios*

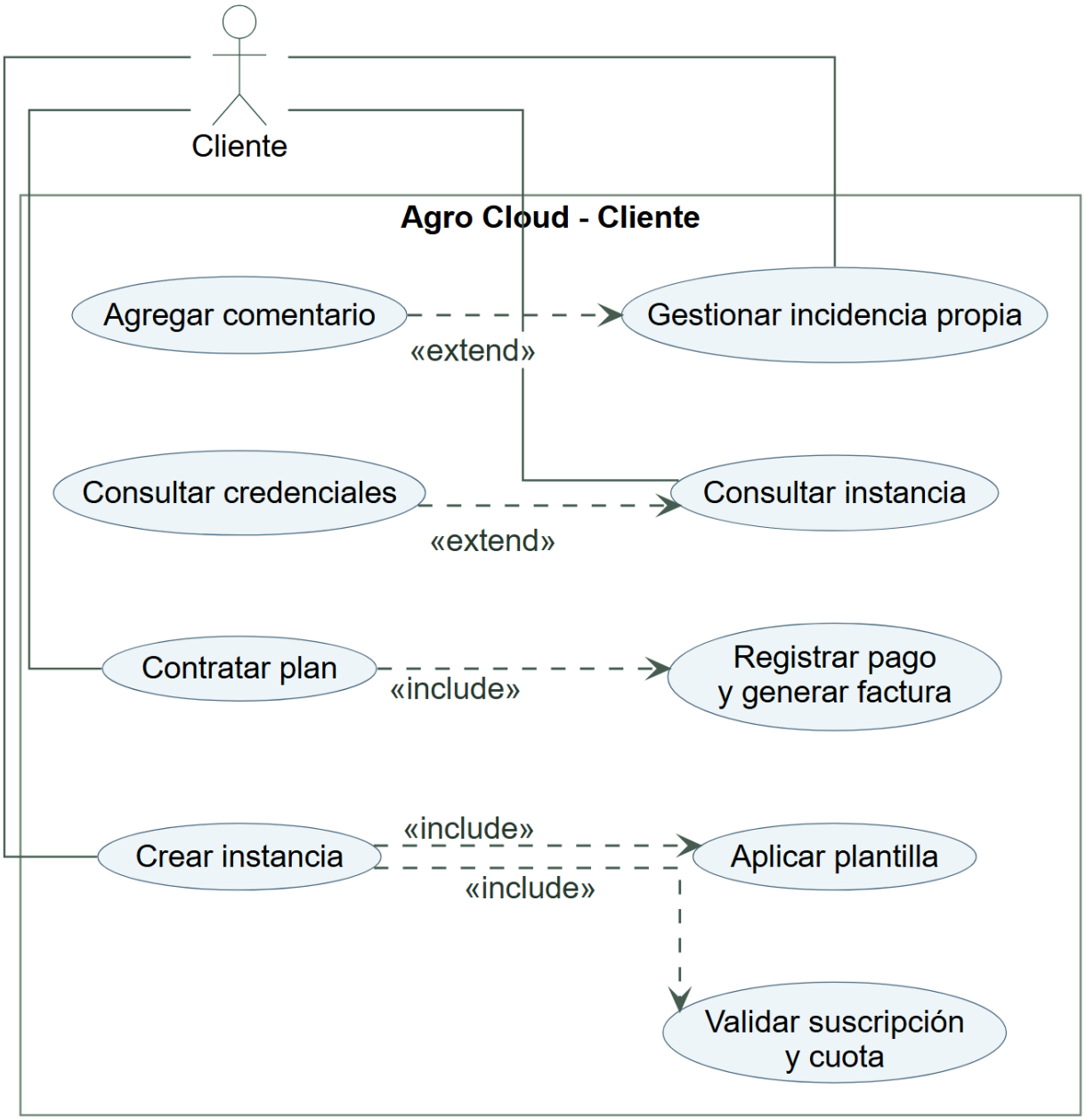


Figura 5. *AgroCloud - Acceso de soporte técnico a modales*

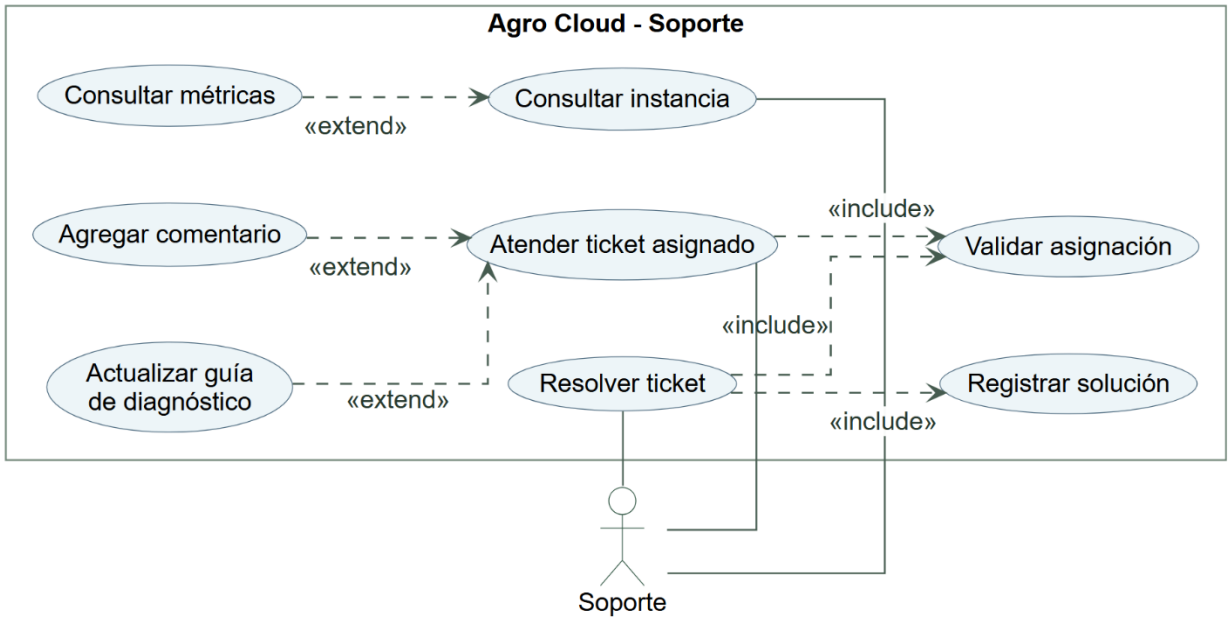


Figura 6. *AgroCloud - Funciones de administrador*

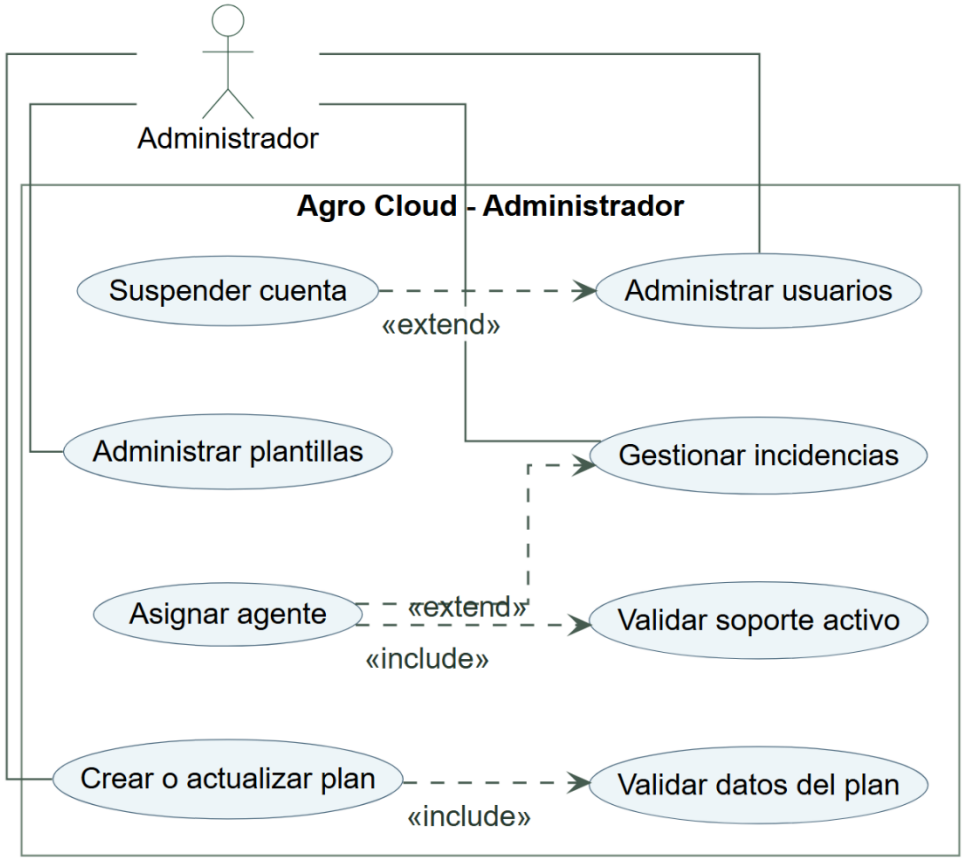


Figura 7. *Registro y contratación de servicios*

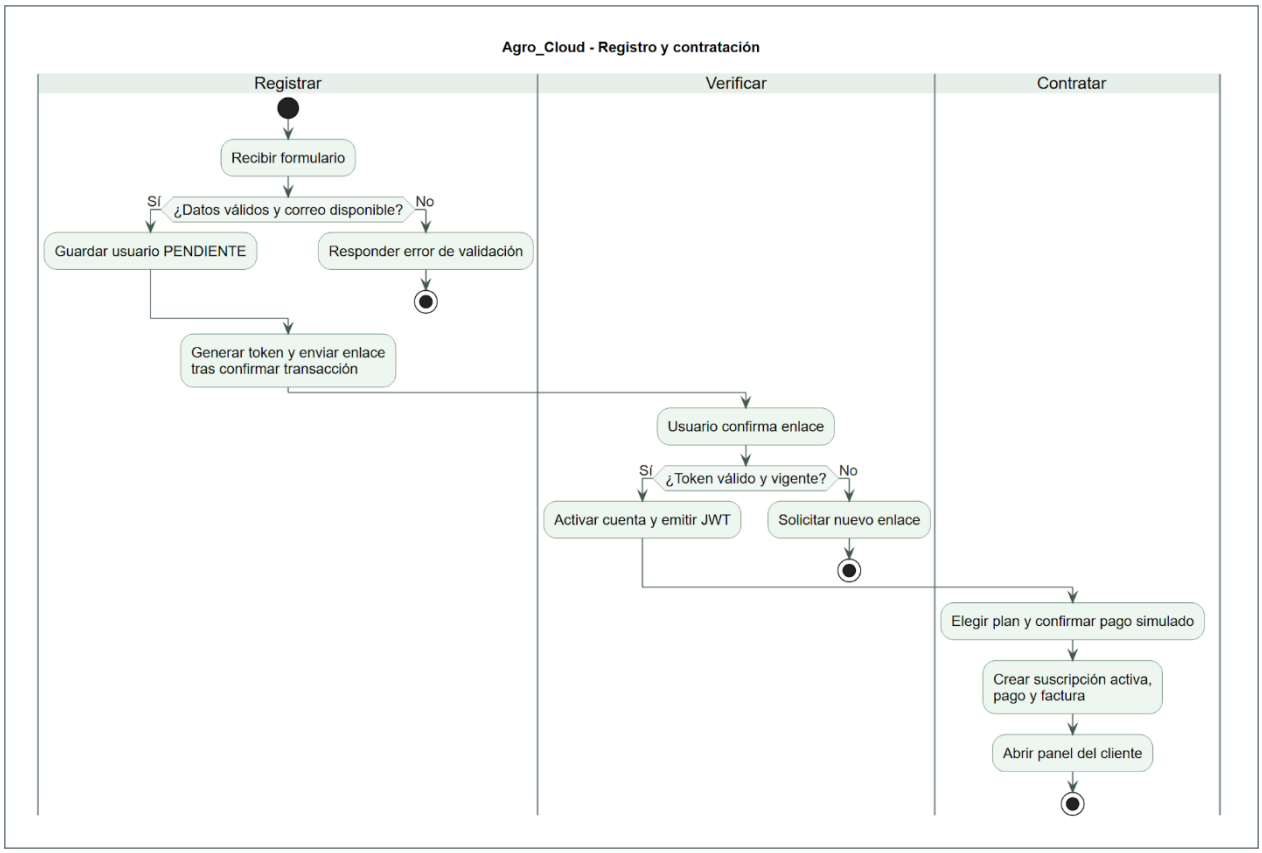


Figura 8. *Creación de una instancia*

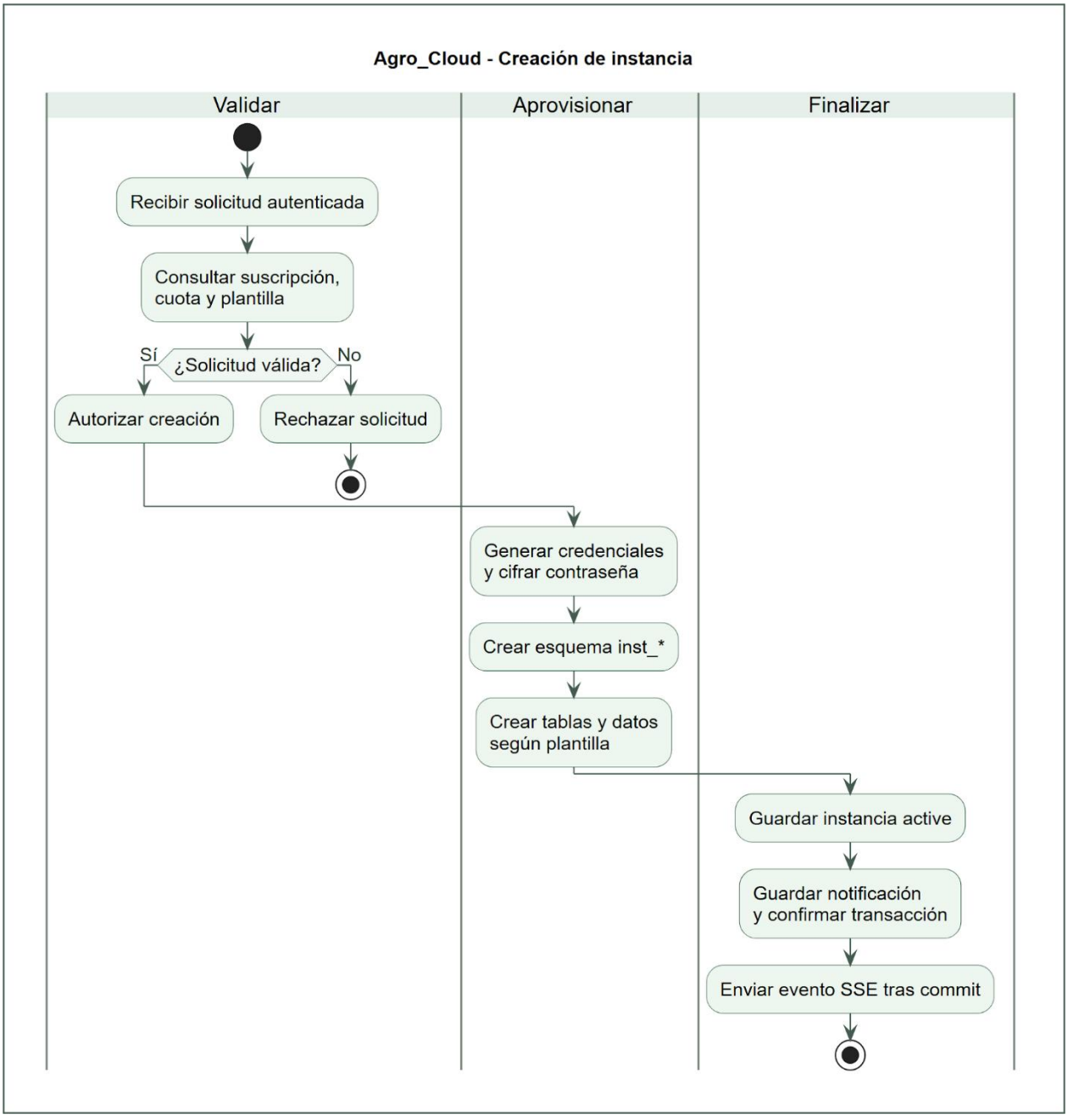


Figura 9. *Atención de una incidencia por parte de soporte técnico*

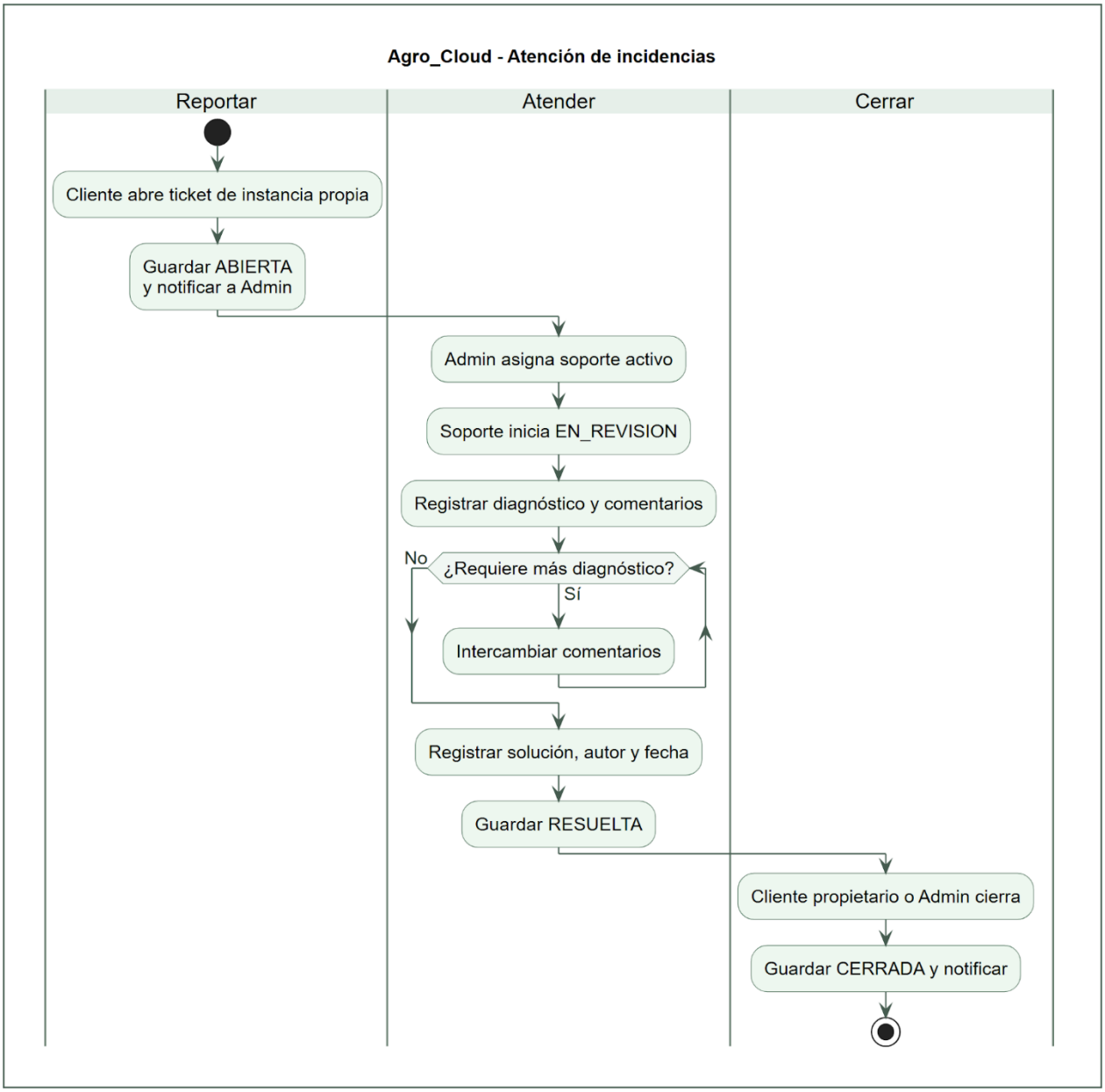


Figura 10. *Inicio de sesión*

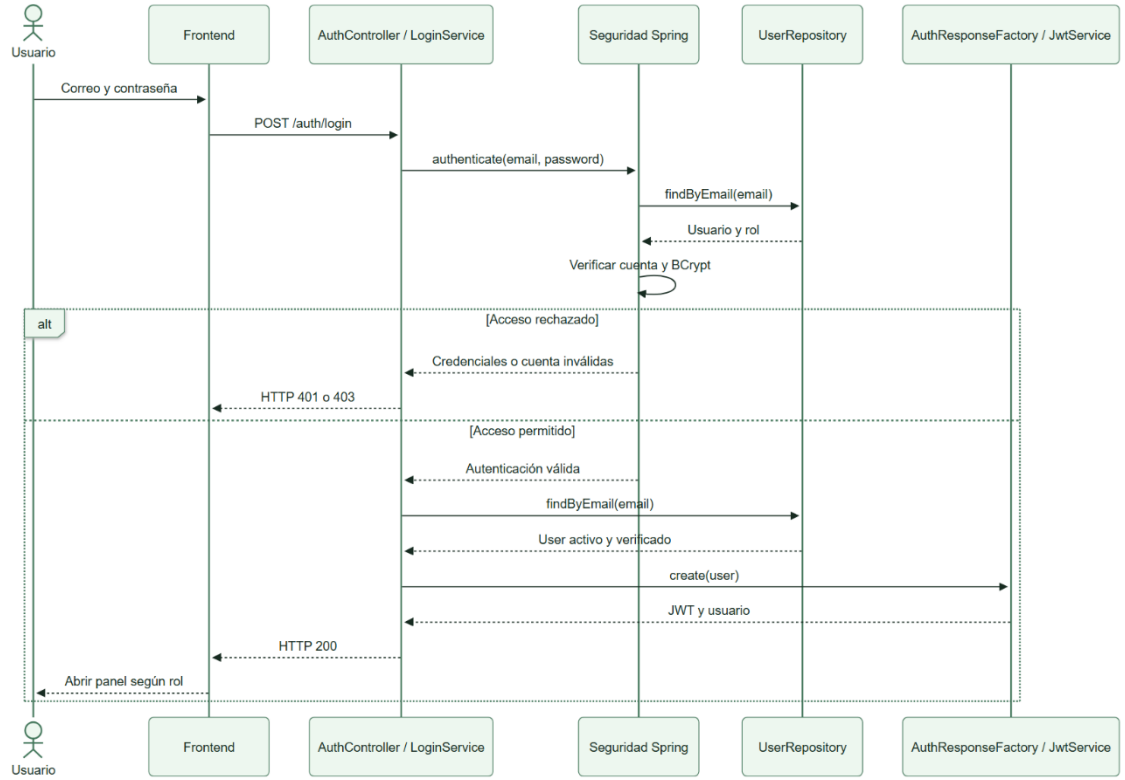


Figura 11. *Aprovisionamiento de instancia*

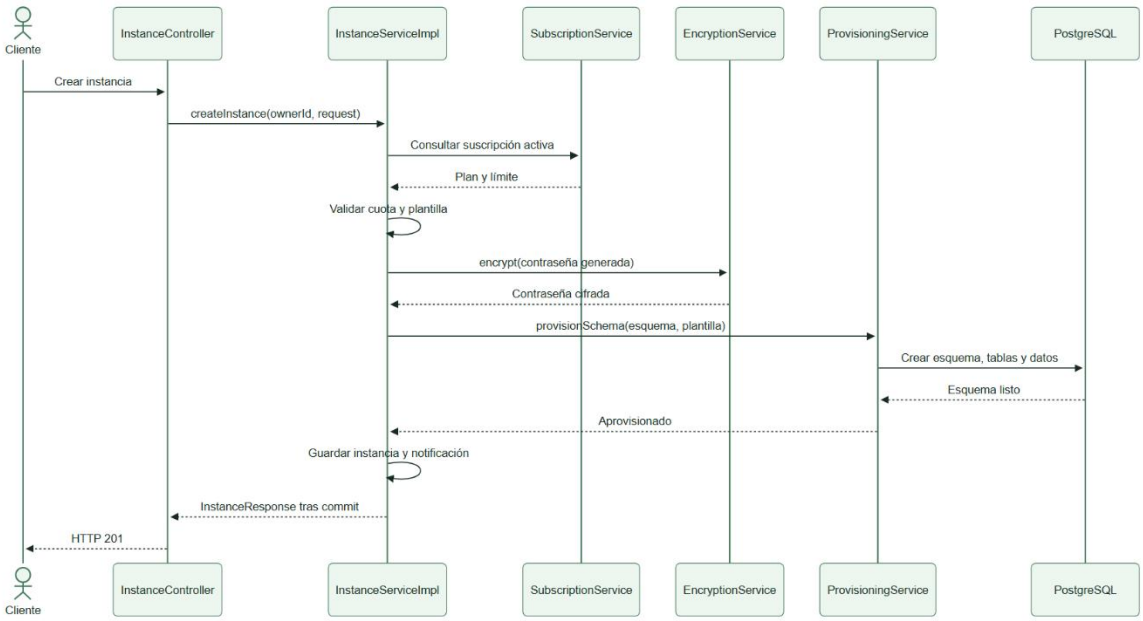


Figura 12. *Soporte y notificaciones*

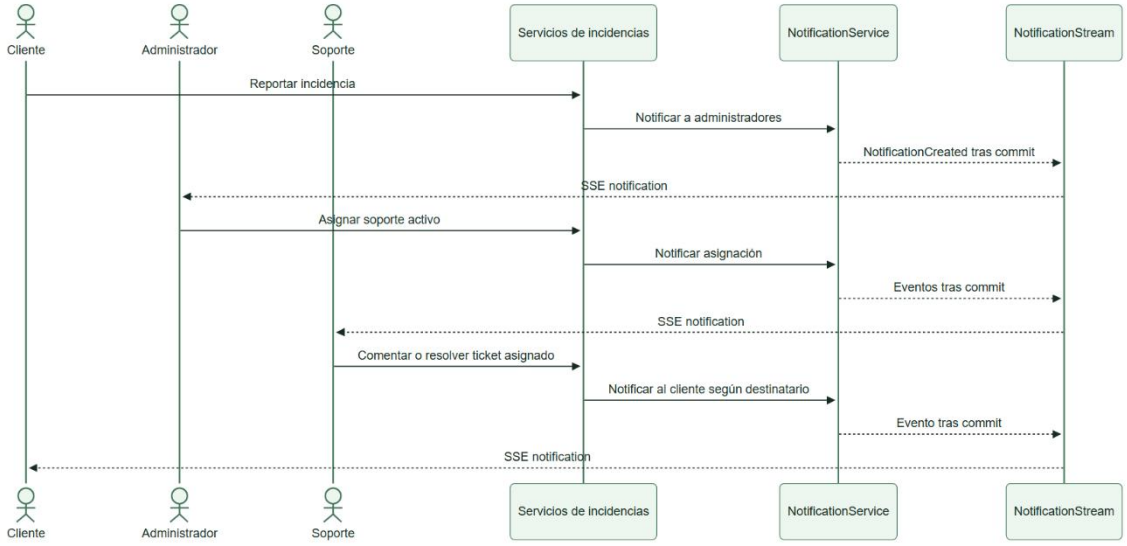


Figura 13. *Transacciones principales de una instancia*

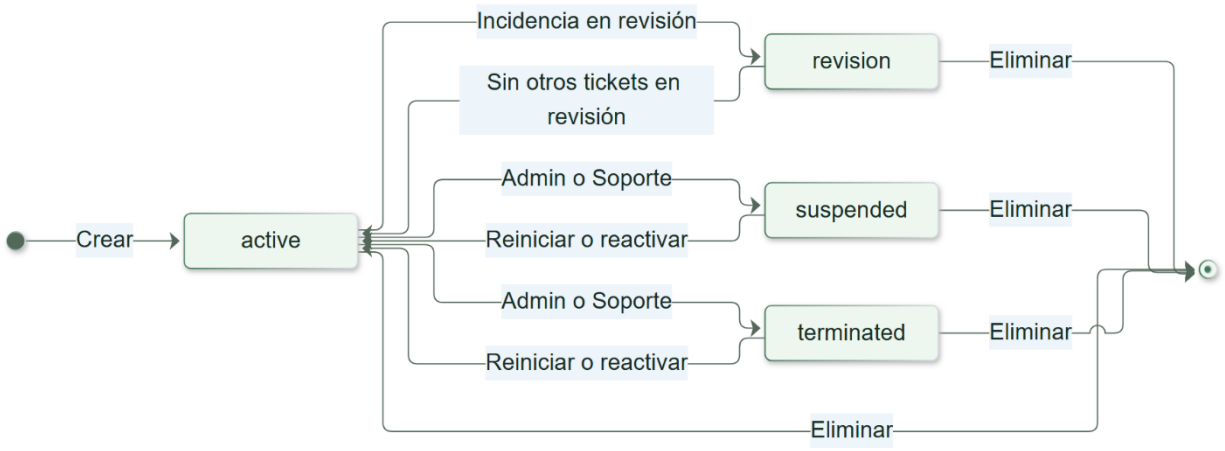


Figura 14. *Estados de una incidencia*

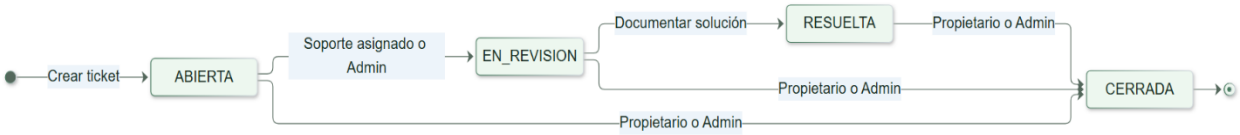


Figura 15. *Componentes principales del sistema*

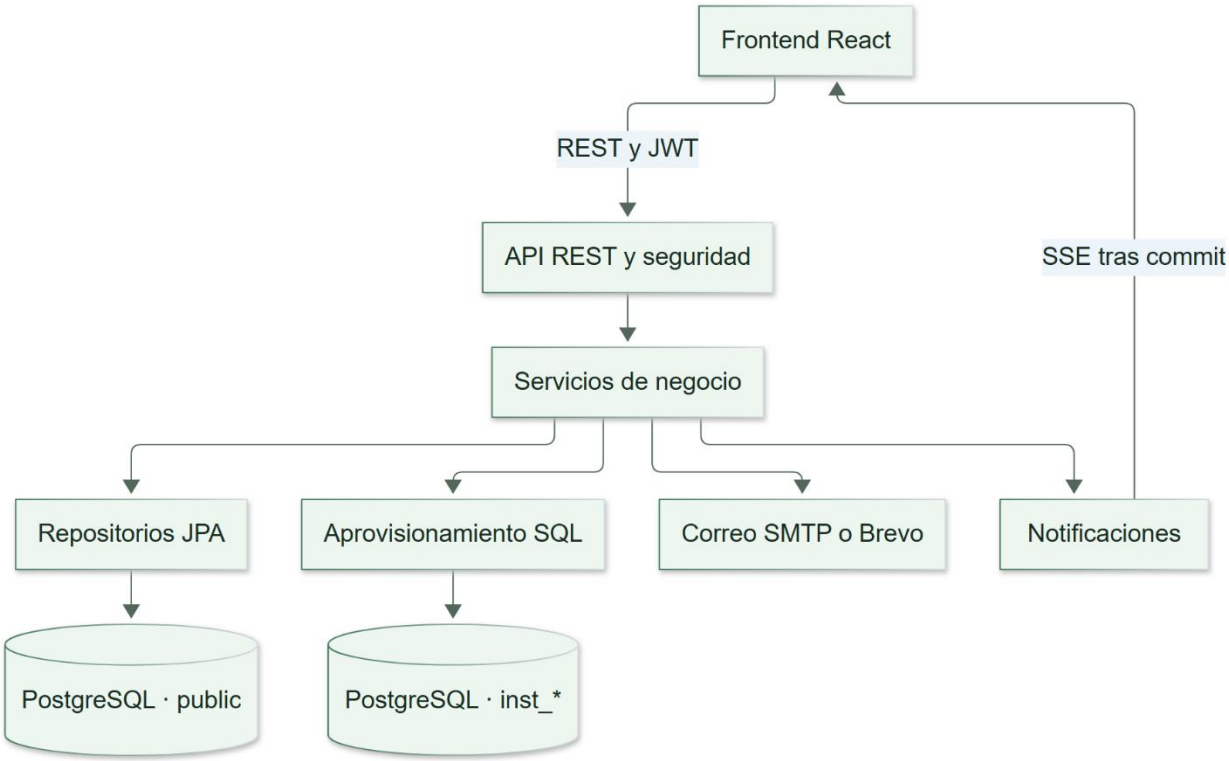


Figura 16. *Despliegue en Docker*

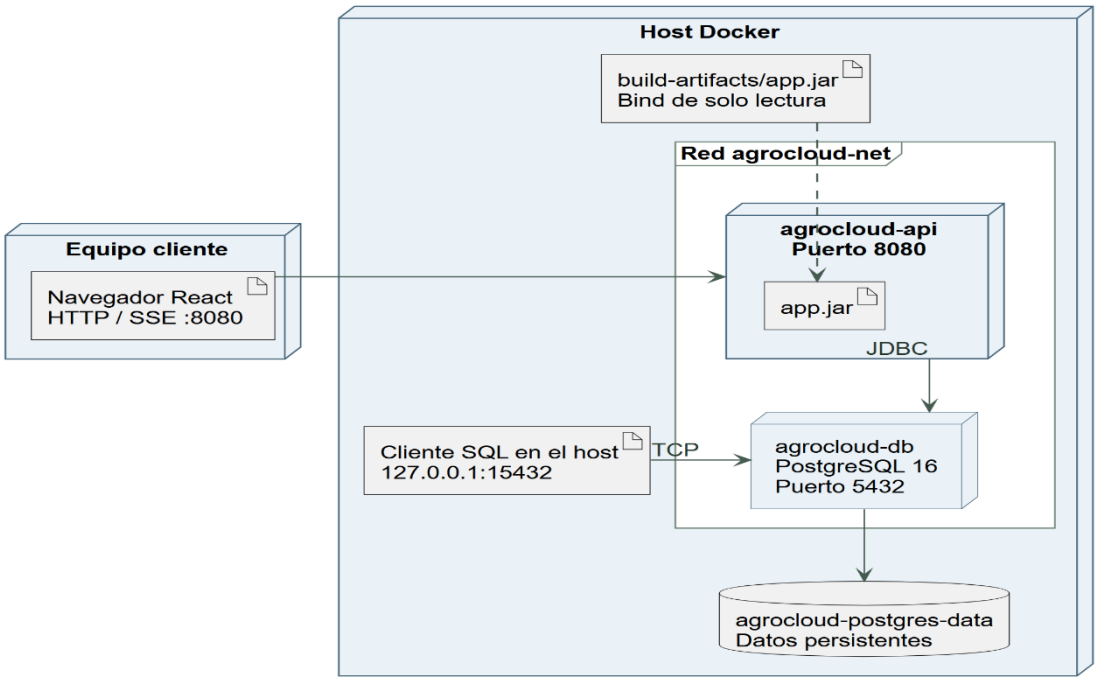


Figura 17. *Directorio del backend de AgroCloud*

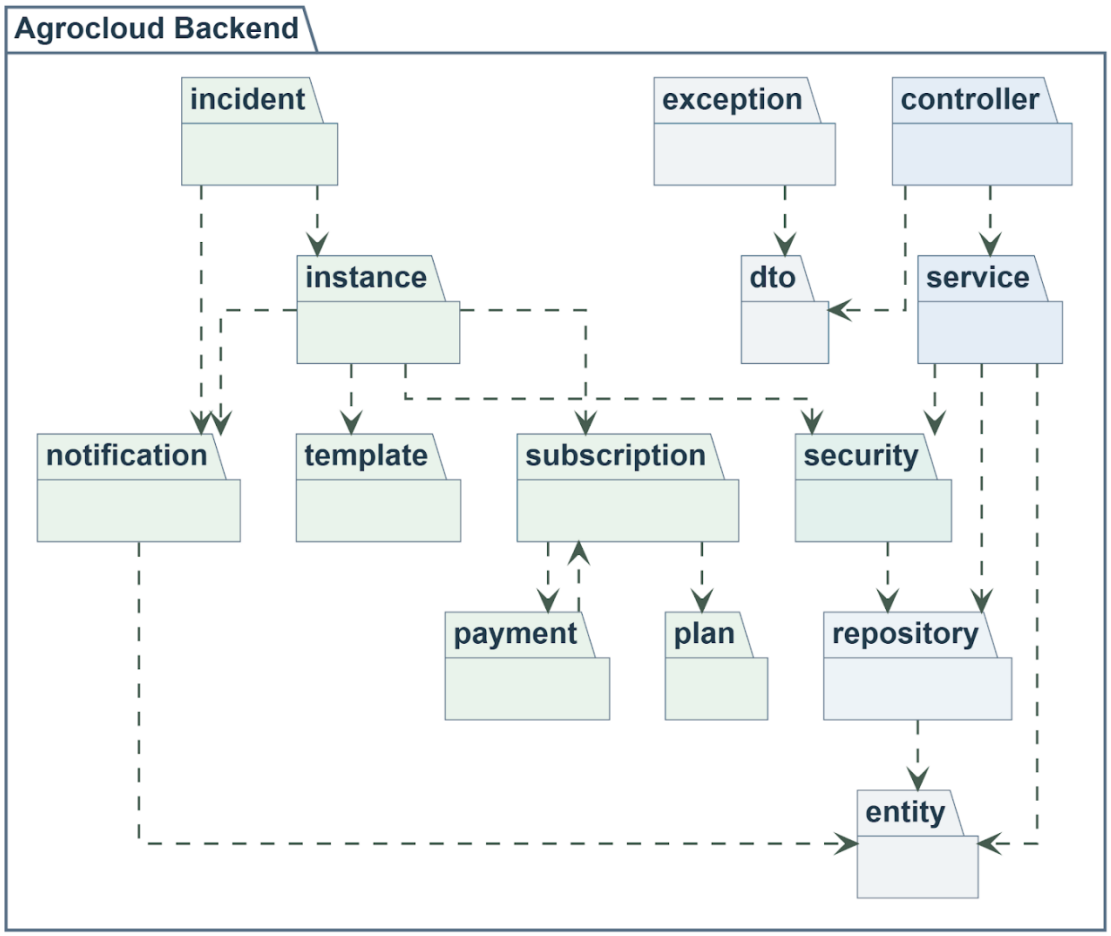


Figura 18. *Modelo Entidad-Relación*

